

Sistema gestión de citas para la Red de servicios Médicos de Piedrazul

Descripción de la Arquitectura de Software

Autores:

*Sofía Moreno Trullo
Liz Yurany Castiblanco Sierra
Yeimy Damaris Joaqui Joaqui
Juliana Andrea Urbano*

Historial de Revisión

Versión	Autores	Fecha
0.1		Mayo 2023
0.31	<i>Francisco Obando</i>	Mayo 2023

Tabla de Contenido

Página

Página	3
1 Introducción	5
2 Descripción del problema	5
3. Arquitectura del Sistema	7
4 Metas y Restricciones de la Arquitectura	16
4.1 Stakeholders y Sus Intereses.....	16
4.2 Metas Arquitectónicas	16
4.3 Restricciones de la Arquitectura	17
5 Vista Lógica	29
5.1 Parte Estructural: descomposición modular	29
5.1.1 Primer Nivel de Refinamiento.....	30
5.1.2 Segundo Nivel de Refinamiento	31
5.1.2.1 Módulo de Autenticación	31
5.1.2.2 Módulo de Citas	32
5.1.2.3 Módulo de Médicos	33
5.1.2.4 Módulo de Pacientes.....	34
5.1.2.5 Módulo de Configuración	34
5.1.2.6 Capa de Persistencia.....	35
5.2 Parte Dinámica (Comportamiento)	35
5.2.1 Escenario de Autenticación	36
5.2.2 Escenario de Gestión de Citas.....	36
5.2.3 Escenario de Exportación CSV.....	37
5.3 Organización de los Componentes.....	38
5.4 View Packets (Catálogo de Componentes).....	39
6 Vista de Despliegue	41
6.1 Nodos y Componentes Desplegados.....	41
6.2 Descripción del Flujo de Despliegue	41
6.3 Tecnología Requerida	42
7 Rationale	43
7.1 Objetivos Arquitectónicos	43

7.2 Decisiones Arquitectónicas Principales.....	43
7.2.1 Elección del estilo Cliente-Servidor	43
7.2.2 Elección de una SPA en React	43
7.2.3 Elección de NestS para el backend.....	44
7.2.4 Autenticación con JWT local.....	44
7.2.5 Prisma como capa de persistencia	44
7.2.6 Base de datos SQLite en el estado actual.....	45
7.2.7 Despliegue separado de frontend y backend.....	45
7.3 Alternativas Consideradas.....	45
7.4 Trade-offs o Compromisos	45
7.5 Consideraciones del Entorno.....	46
7.6 Decisiones Futuras y Evolución.....	46
7.7 Justificación Final.....	46

Sistema gestión de citas para la Red de Servicios Médicos de Piedrazul

Architecture Document – SAD

1 Introducción

El presente documento tiene como propósito describir la arquitectura del sistema de software desarrollado para la gestión de citas médicas en un centro de atención en salud. Este documento, denominado Software Architecture Document (SAD), establece una visión integral de la estructura del sistema, sus componentes principales, las decisiones arquitectónicas adoptadas y la forma en que estos elementos interactúan para satisfacer los requerimientos funcionales y no funcionales definidos.

Asimismo, el documento sirve como guía para el diseño, desarrollo, implementación, despliegue y mantenimiento del sistema, facilitando la comprensión de su funcionamiento y evolución. También permite asegurar la alineación entre los objetivos del negocio, las necesidades de los usuarios y las soluciones tecnológicas propuestas, promoviendo la calidad, escalabilidad, seguridad y trazabilidad del sistema. Este documento está dirigido a los diferentes actores involucrados en el desarrollo y uso del sistema, incluyendo principalmente al equipo técnico (desarrolladores, arquitectos de software y testers), quienes requieren una comprensión detallada de la arquitectura para implementar y mantener la solución. Sin embargo, se considera audiencia a los stakeholders no técnicos como gerentes de proyecto, docentes evaluadores y destinatarios del sistema, quienes necesitan una visión general de la estructura y funcionamiento del sistema para la toma de decisiones, seguimiento del proyecto y validación de los requerimientos establecidos.

2 Descripción del problema

Contexto y antecedentes: Actualmente, el proceso de gestión de citas médicas en el entorno objetivo se realiza mediante herramientas tradicionales, incluyendo software de escritorio con funcionalidades limitadas o procesos manuales que dificultan la administración eficiente de la información. Estas soluciones presentan restricciones en términos de accesibilidad, escalabilidad y trazabilidad, lo que limita su capacidad para adaptarse a las necesidades actuales de digitalización. Adicionalmente, no existe un mecanismo integrado que permita a los pacientes agendar citas de manera autónoma a través de la web, ni una adecuada centralización de la información clínica y administrativa, lo que genera dependencia de intermediarios y aumenta la carga operativa del personal.

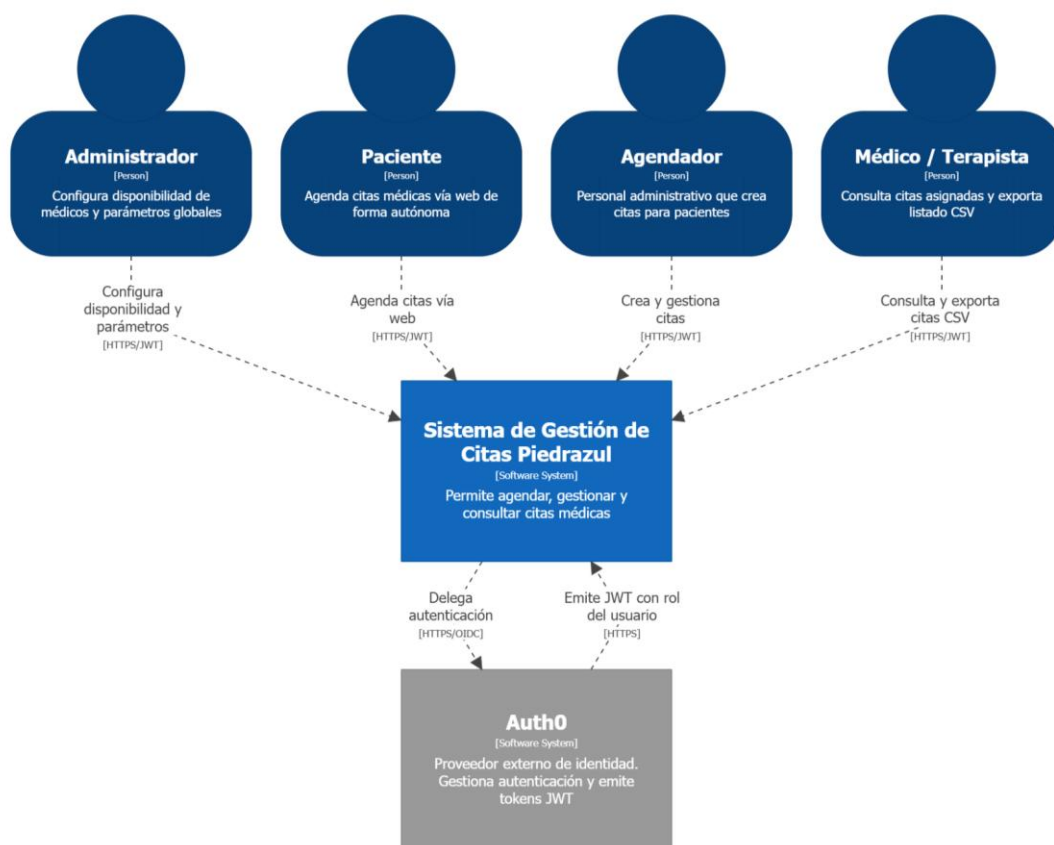
Necesidades y objetivos: El sistema propuesto busca satisfacer la necesidad de contar con una plataforma web centralizada que permita gestionar de manera eficiente las citas médicas, los usuarios del sistema y la información clínica básica. Entre los objetivos principales se encuentran mejorar la eficiencia operativa, reducir errores en la asignación de citas, facilitar el acceso de los pacientes a los servicios de salud mediante agendamiento en línea y garantizar la seguridad y confidencialidad de la información. Asimismo, se pretende proporcionar herramientas de apoyo a la toma de decisiones mediante reportes estadísticos, así como mecanismos de auditoría que aseguren la trazabilidad de todas las acciones realizadas dentro del sistema.

Problemas y desafíos actuales: En el entorno actual se identifican diversos problemas, entre los que se destacan la duplicación de citas, la falta de control sobre la disponibilidad de los profesionales, la ausencia de trazabilidad en modificaciones de agendas y la limitada capacidad de acceso remoto al sistema debido a la falta de conectividad continua a redes.

WIFI. Además, la gestión manual o semi-automatizada incrementa la probabilidad de errores humanos y dificulta la escalabilidad del servicio. Otro desafío relevante es la ausencia de mecanismos robustos de seguridad y autenticación, lo cual pone en riesgo la integridad y confidencialidad de los datos clínicos.

Limitaciones y restricciones: El desarrollo del sistema se encuentra sujeto a ciertas restricciones, entre las que se incluyen el uso de tecnologías específicas como mecanismos de autenticación basados en JWT mediante Auth0, la implementación de pruebas de seguridad utilizando herramientas como OWASP ZAP, y el cumplimiento de lineamientos académicos relacionados con integración continua, despliegue en la nube y documentación arquitectónica. Adicionalmente, se deben considerar limitaciones de recursos, tales como el uso preferente de servicios gratuitos para el despliegue, así como restricciones de tiempo propias del desarrollo académico.

Impacto del problema: Los problemas identificados afectan directamente la calidad del servicio ofrecido a los pacientes, generando demoras, inconsistencias en la asignación de citas y dificultades en el acceso a la atención médica. Para el personal administrativo y médico, estas deficiencias implican una mayor carga operativa, falta de control sobre la información y limitaciones en la gestión eficiente de la agenda. La implementación de un sistema adecuado permitirá mejorar significativamente la organización del servicio, optimizar los procesos internos, garantizar la seguridad de la información y proporcionar una mejor experiencia a los usuarios finales, contribuyendo así a la transformación digital del entorno de atención en salud.



System Context View: Sistema de Gestión de Citas Piedrazul

Diagrama de Contexto - Sistema de Gestión de Citas Piedrazul
domingo, 3 de mayo de 2026, 10:16 a. m. hora estándar de Colombia

3. Arquitectura del Sistema

La arquitectura del sistema de gestión de citas médicas de Piedrazul está representada siguiendo el enfoque del framework 4+1, junto con las recomendaciones del Proceso Unificado y el método ADD (Attribute-Driven Design). Este enfoque permite describir el sistema desde múltiples perspectivas, facilitando la comprensión de su estructura, comportamiento y despliegue. Las vistas incluidas en esta versión del documento son:

- **Vista de Casos de Uso y Escenarios de Calidad:** Describe los casos de uso más significativos del sistema, identificando los actores principales como paciente, agendador, médico/terapista y administrador, así como sus interacciones con el sistema. De igual forma, se describen los escenarios de calidad más relevantes, incluyendo aspectos como seguridad, concurrencia y disponibilidad

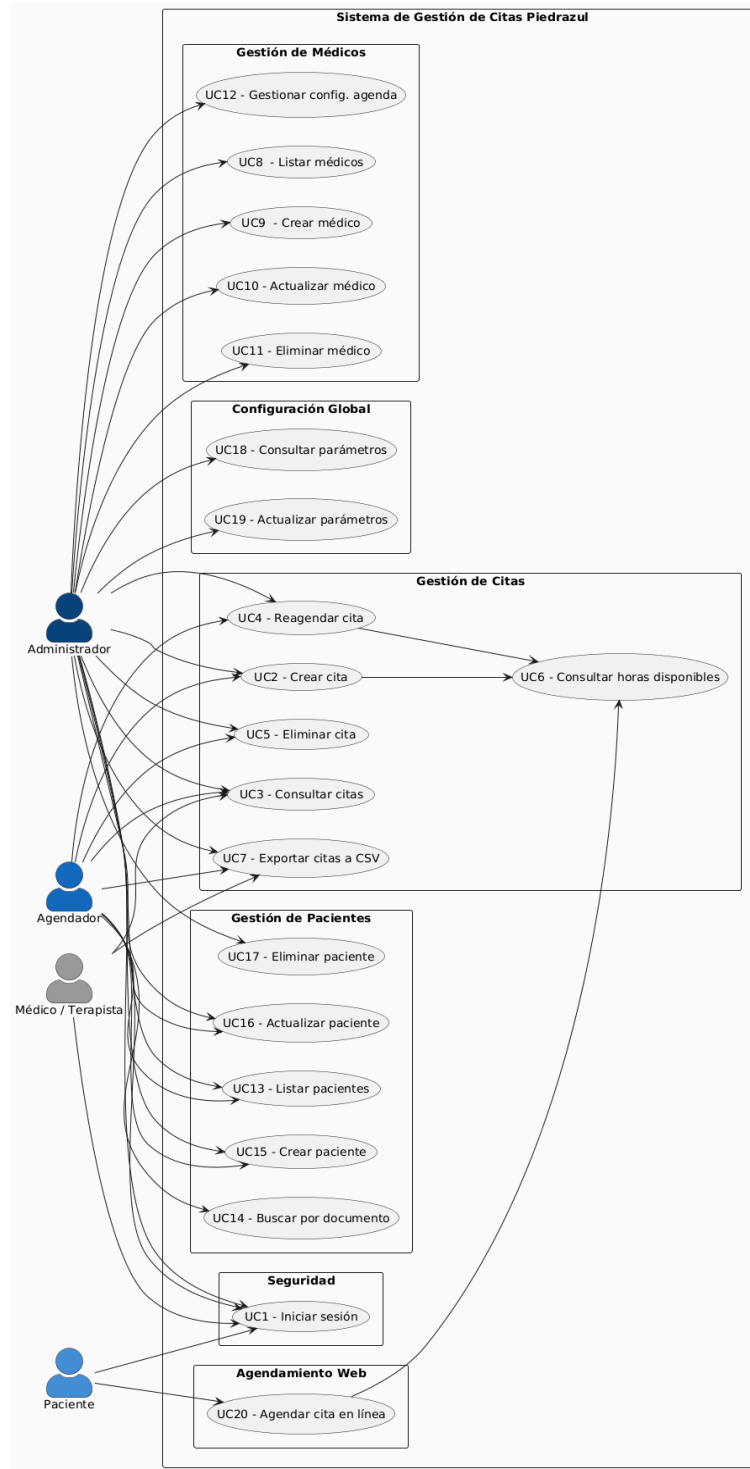


Diagrama de casos de uso : Agendamiento de cita

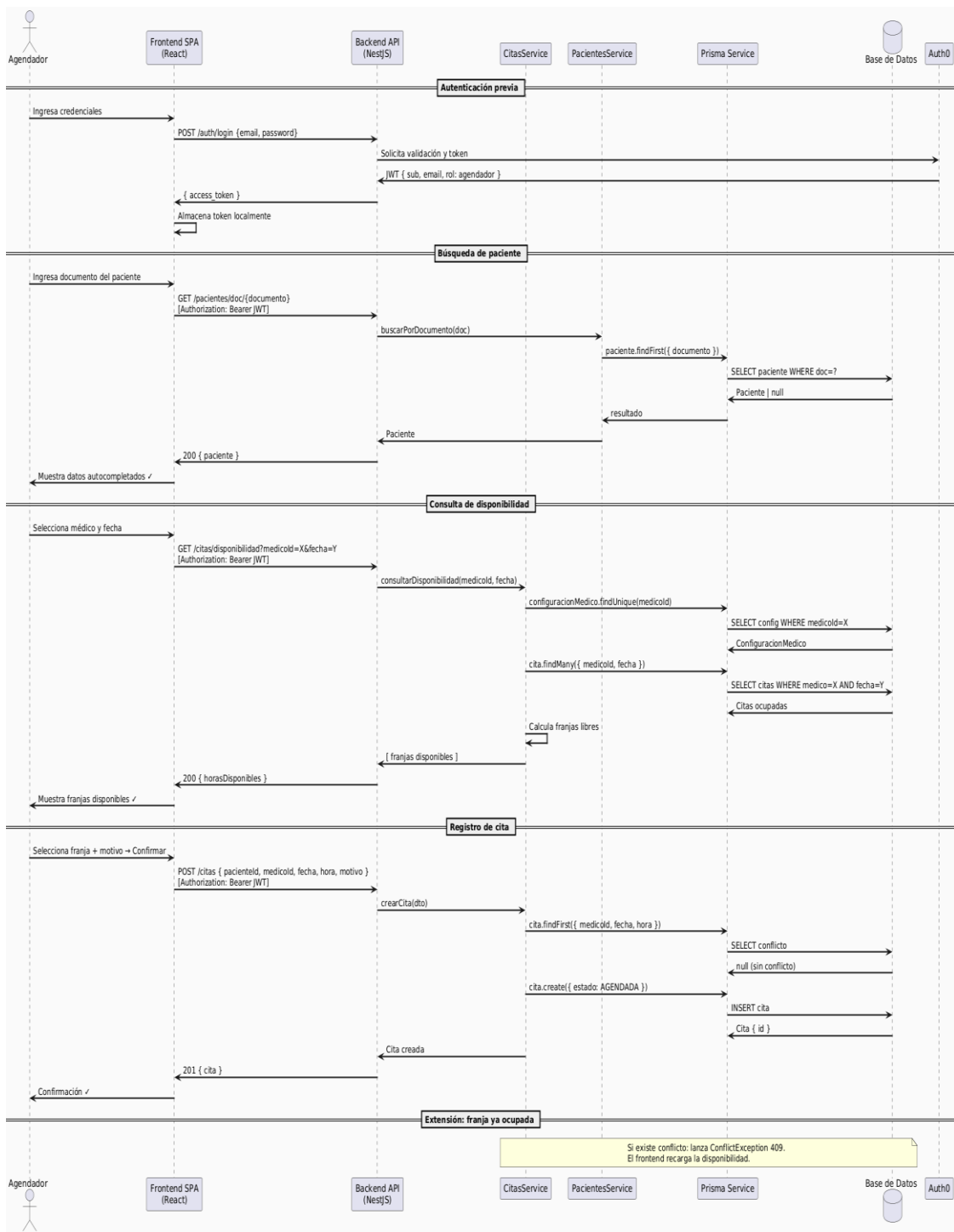
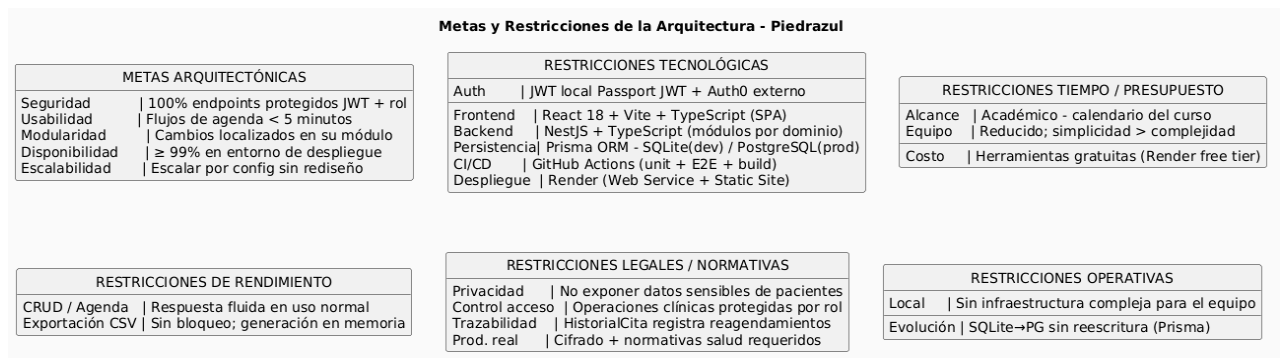


Diagrama de secuencia- Agendamiento de cita

- Vista de Metas y Restricciones:** Describe las restricciones tecnológicas, normativas y de entorno que influyen en las decisiones arquitectónicas del sistema. Entre estas se incluyen la autenticación mediante JWT con Key Cloak, la implementación de pruebas de seguridad con OWASP ZAP, el uso de integración y despliegue continuos (CI/CD), y el despliegue en la nube utilizando tecnologías gratuitas.



- Vista Lógica:** Describe la arquitectura del sistema a nivel estructural, identificando los principales módulos y sus responsabilidades. El sistema se organiza en capas, incluyendo la capa de presentación (interfaz web), la capa de lógica de negocio (servicios de citas, usuarios, auditoría y reportes) y la capa de datos (base de datos). Asimismo, se incluye la integración con un servicio externo de autenticación (Keycloak).

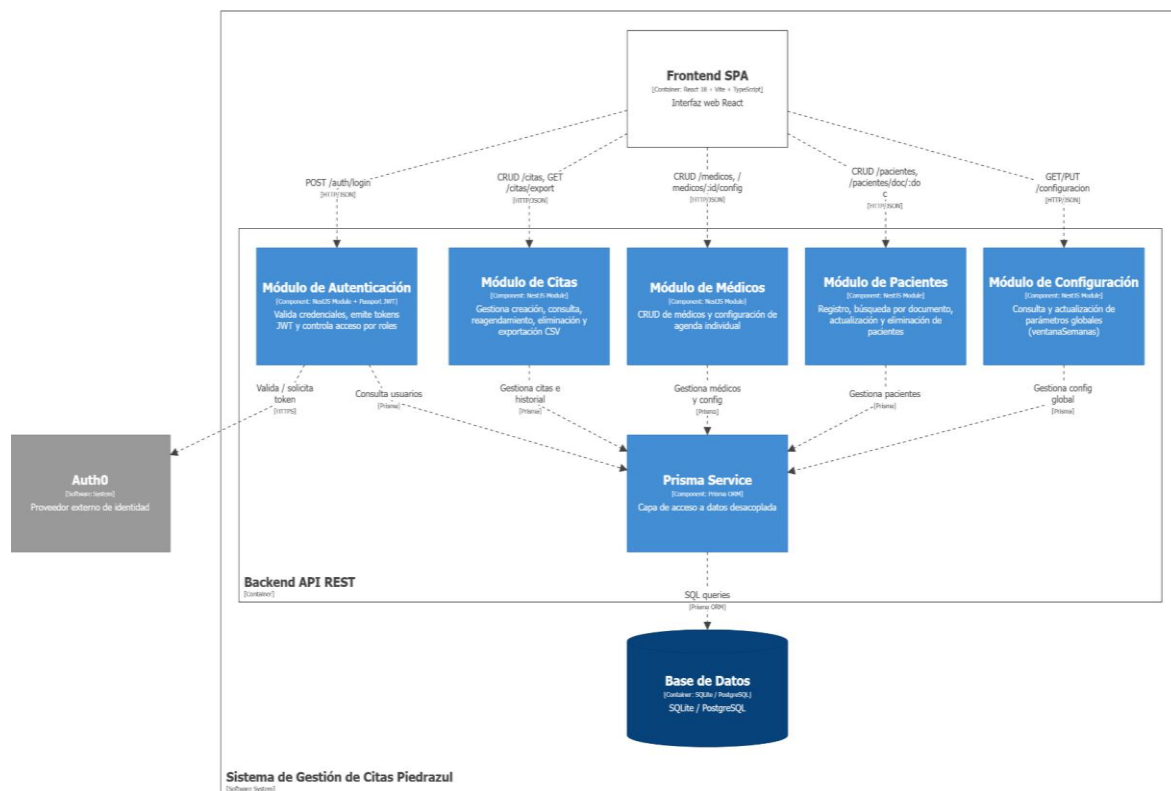


Diagrama de componentes del Backend Nest.js

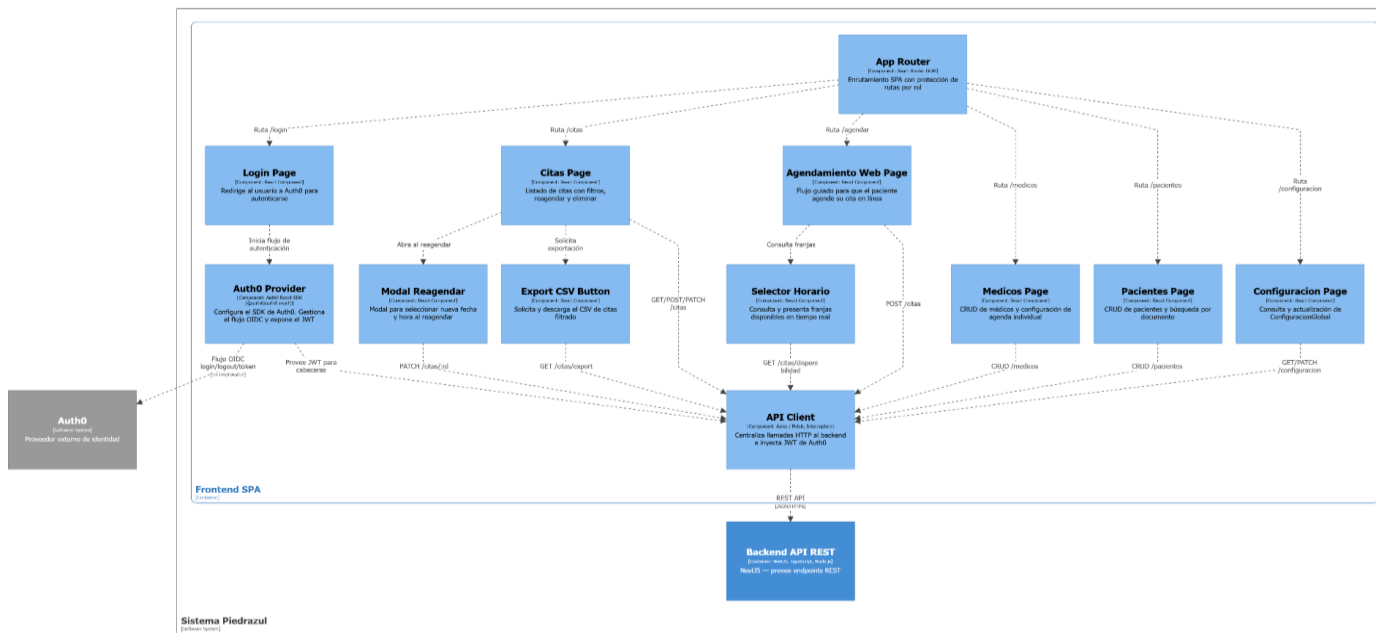


Diagrama de componentes Frontend SPA

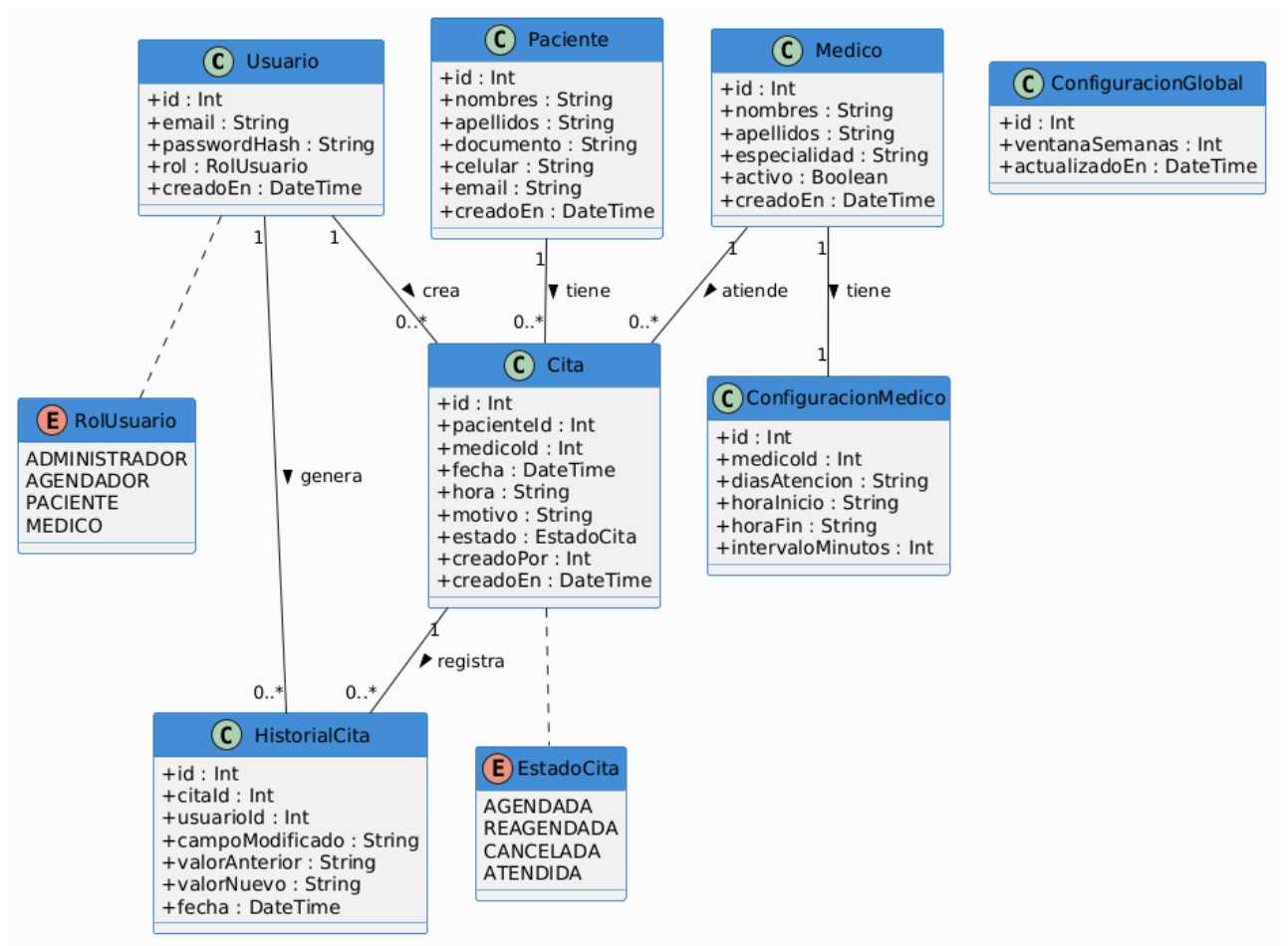
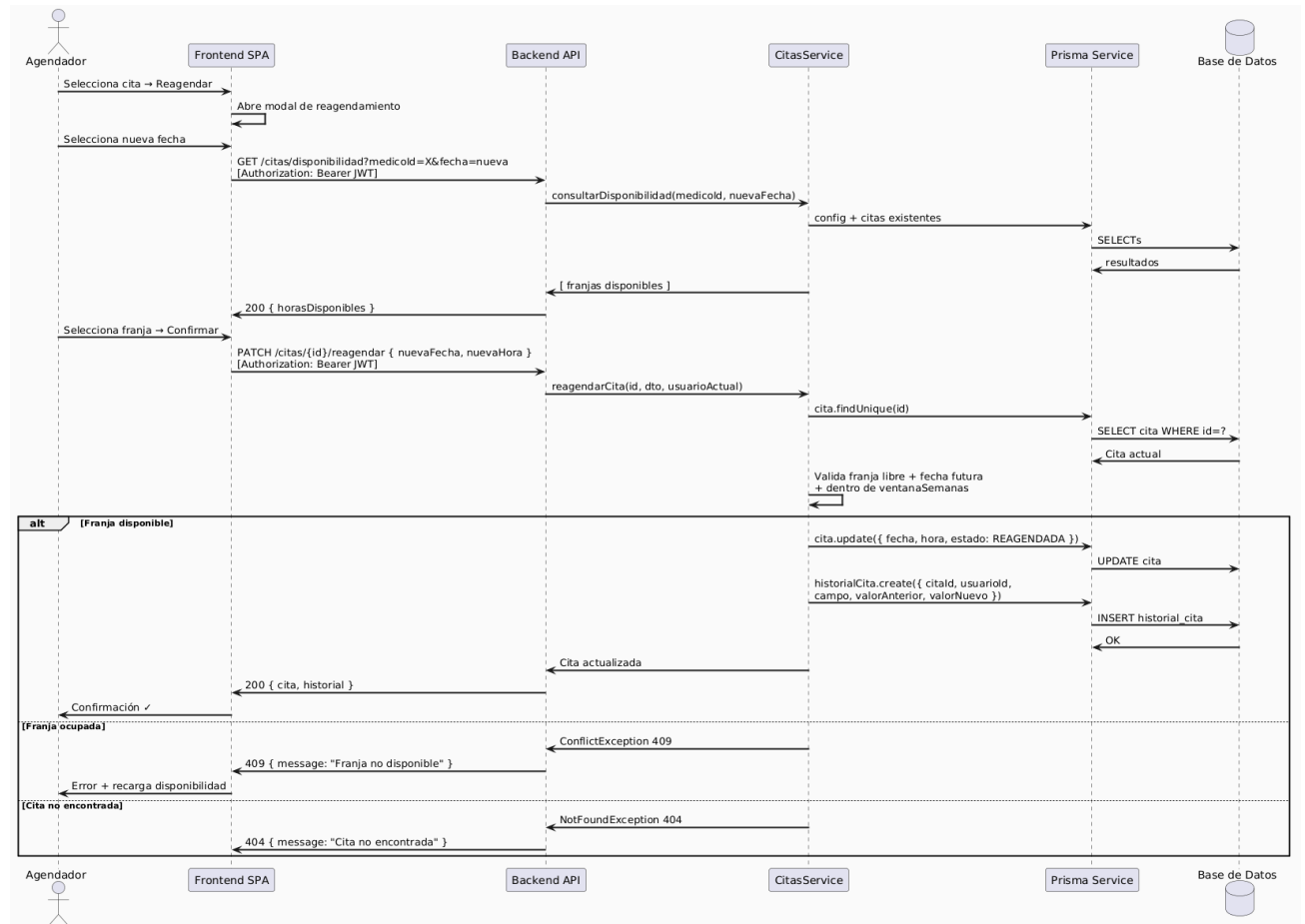
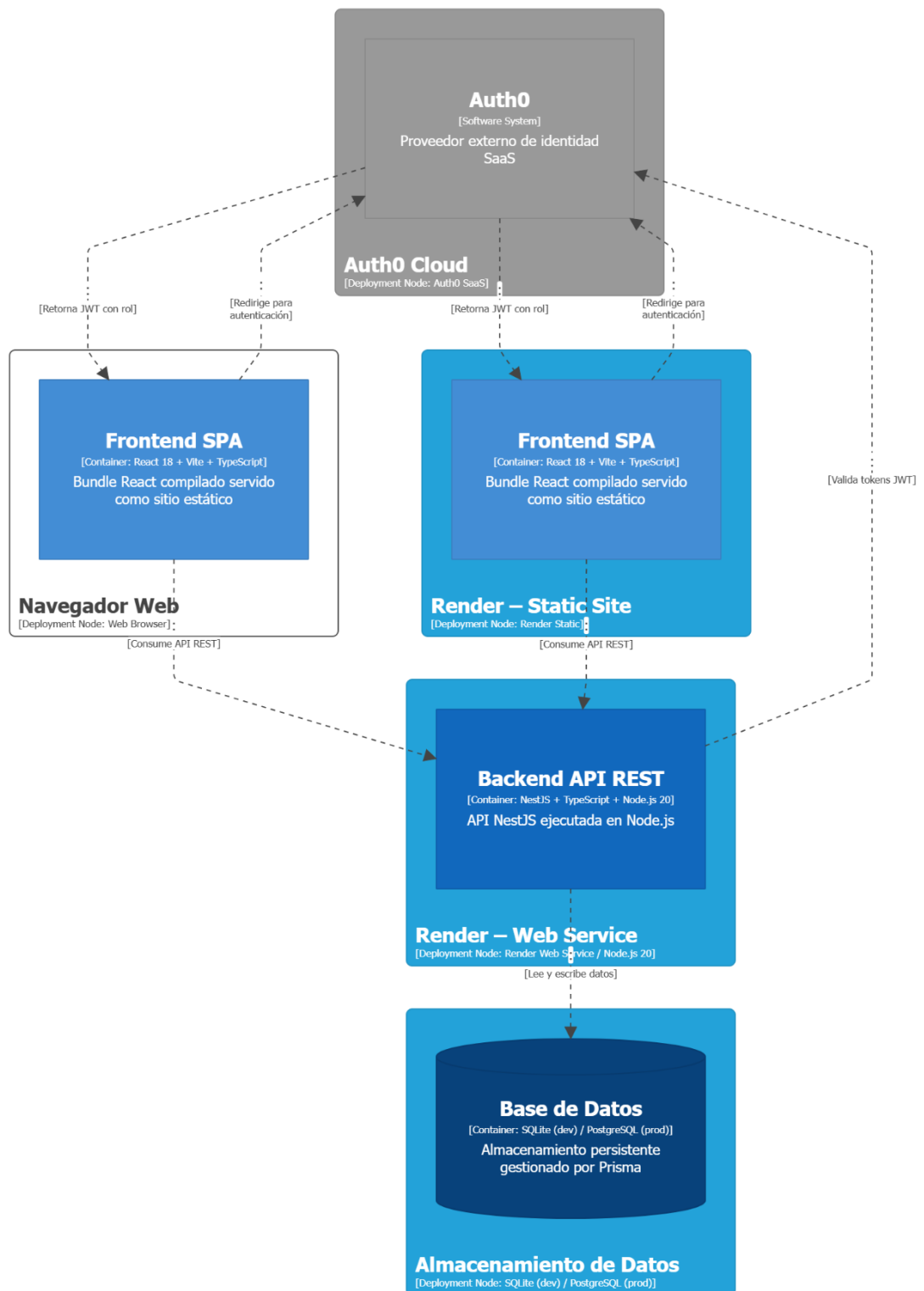


Diagrama de clases

- Vista de Procesos:** Describe los procesos dinámicos del sistema, incluyendo la interacción entre los diferentes componentes durante la ejecución de funcionalidades clave, como el agendamiento de citas, el re-agendamiento y la autenticación de usuarios. Esta vista permite entender la comunicación y sincronización entre los elementos del sistema.



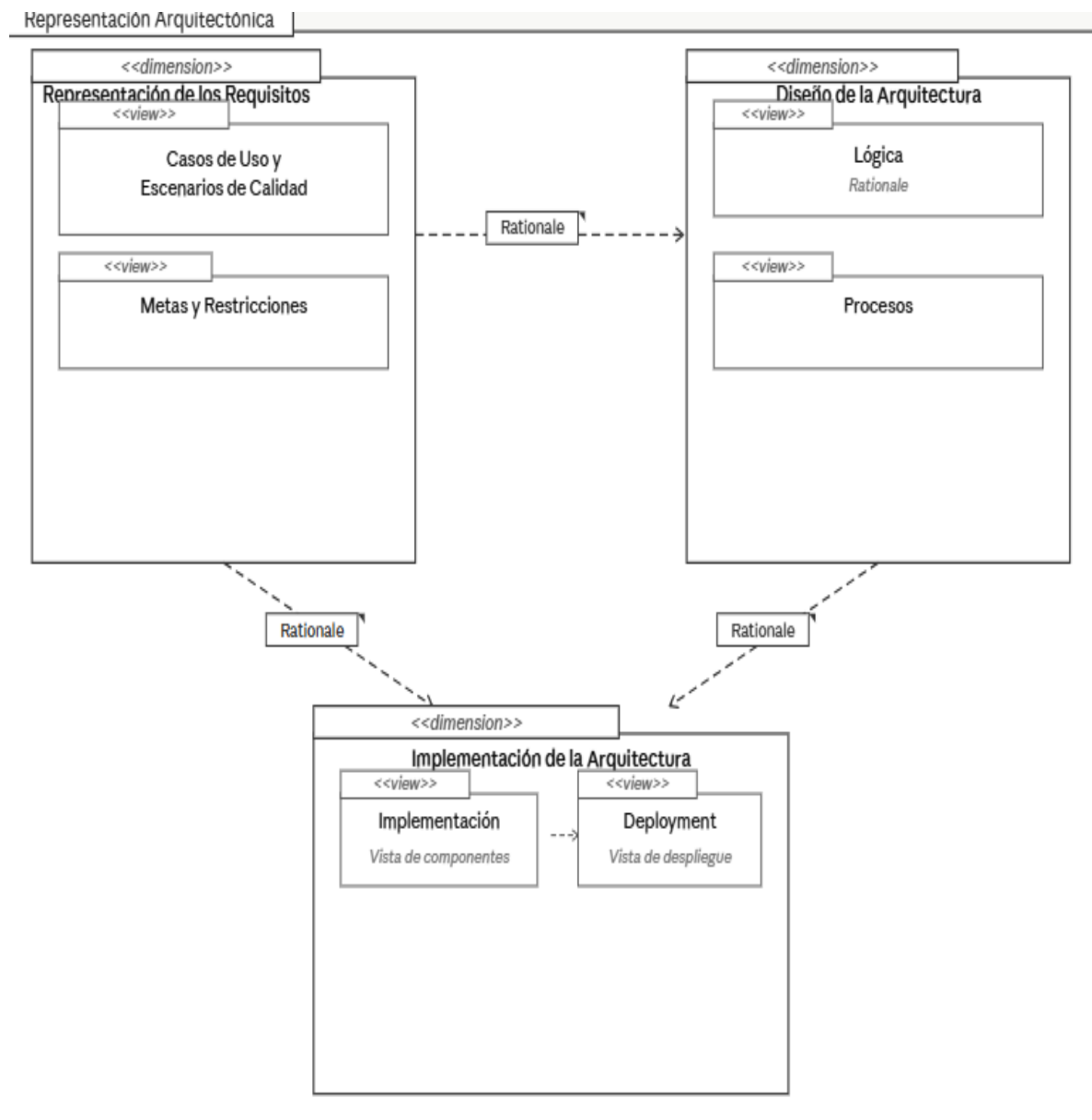
- Vista de Implementación (Despliegue):** Describe la distribución física de los componentes del sistema en la infraestructura tecnológica, incluyendo el cliente web, el servidor de aplicación, el servidor de autenticación (Auth0) y la base de datos. También muestra las relaciones de comunicación entre estos elementos.



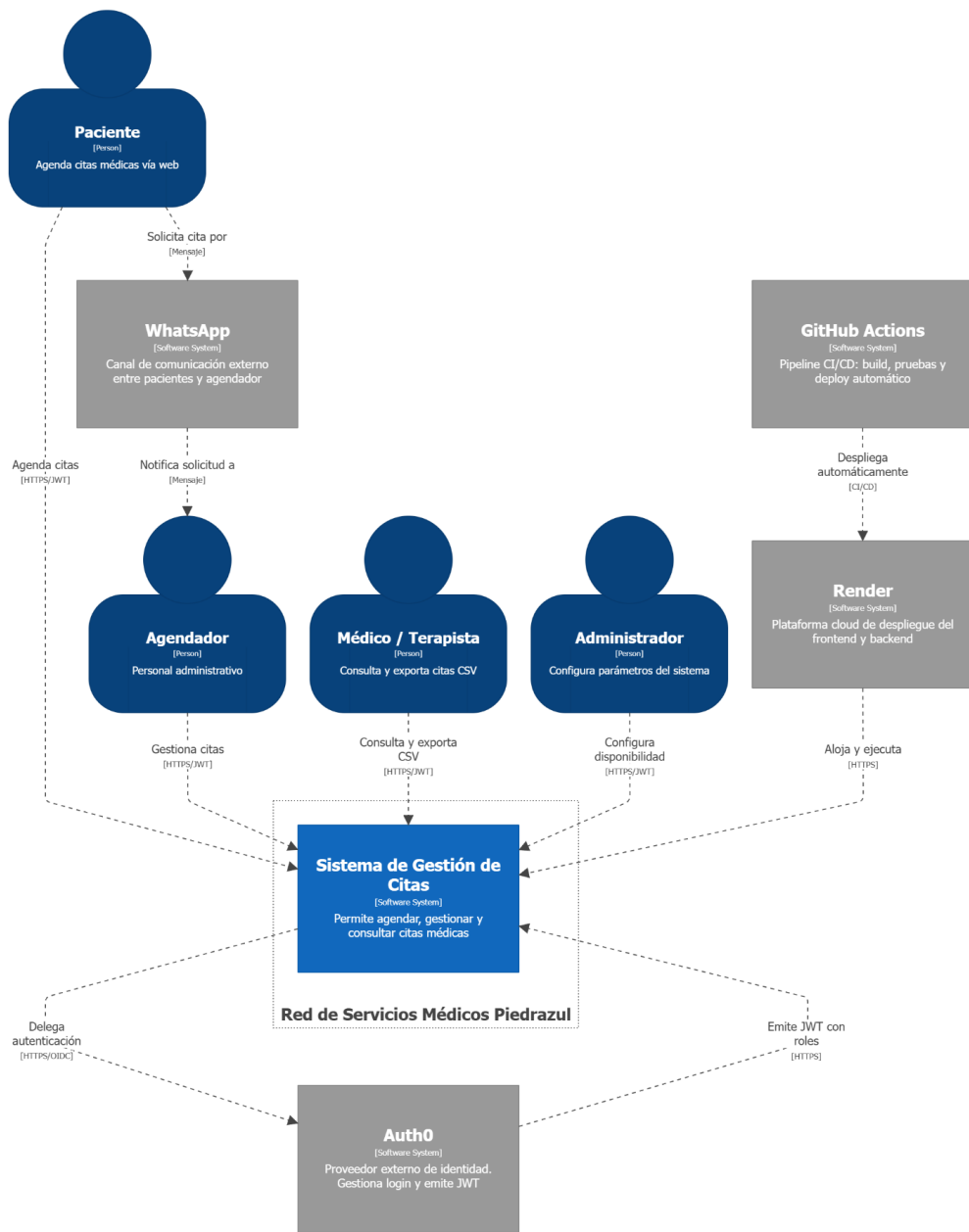
C4 – Vista de Despliegue en Render + Auth0
domingo, 3 de mayo de 2026, 5:17 p. m. hora estándar de Colombia

Vista despliegue en Render +Auth0

Las vistas presentadas forman en su conjunto una especificación de la arquitectura del sistema en su estado actual de desarrollo. La relación entre estas vistas se representa mediante un diagrama general que permite visualizar la interacción entre los diferentes componentes y su despliegue en la infraestructura.



Representación arquitectónica



System Landscape View

System Landscape - Red de Servicios Médicos Piedrazul
domingo, 3 de mayo de 2026, 10:10 a. m. hora estándar de Colombia

4 Metas y Restricciones de la Arquitectura

Para definir las metas y restricciones de la arquitectura se tomaron como base los requisitos del sistema, el contexto académico del proyecto y las necesidades expresadas por los interesados identificados. A continuación, se presenta la descripción de los participantes, los atributos de calidad derivados del análisis y las restricciones que condicionan el diseño.

4.1 Stakeholders y Sus Intereses

Los interesados principales del sistema son los siguientes:

- Administrador de la clínica: requiere control total sobre la configuración global del sistema, gestión de médicos y visibilidad operativa sobre las operaciones del sistema.
- Agendador: requiere registrar, consultar, reagendar y exportar citas de forma rápida y confiable, con búsqueda ágil de pacientes por documento.
- Paciente: requiere solicitar y agendar citas de manera simple, sin fricción, a través de un flujo guiado por pasos accesible desde el navegador.
- Equipo técnico del proyecto: requiere una arquitectura mantenible, modular y fácil de desplegar, que permita evolución sin rediseño completo.
- Docente y evaluador académico: requiere trazabilidad entre requisitos, decisiones arquitectónicas y evidencia técnica del sistema implementado.

A partir de estas necesidades se priorizaron los atributos de calidad de Seguridad, Usabilidad, Modificabilidad, Disponibilidad y Escalabilidad.

4.2 Metas Arquitectónicas

Las metas arquitectónicas del sistema son las siguientes:

- Seguridad de acceso: asegurar que solo usuarios autorizados ejecuten operaciones protegidas. La métrica objetivo es que el 100% de los endpoints críticos estén protegidos por autenticación JWT y validación de rol antes de ejecutar cualquier operación.
- Usabilidad operativa: permitir que los flujos frecuentes se completen con baja fricción. La métrica objetivo es que las operaciones comunes de agenda, como crear o reagendar una cita, se completen en menos de cinco minutos por flujo.
- Modularidad y mantenibilidad: separar responsabilidades por dominio funcional para reducir el impacto de cambios. La métrica objetivo es que los cambios de funcionalidad queden localizados en su módulo sin afectar transversalmente el resto del sistema.
- Disponibilidad funcional: mantener continuidad del servicio en escenarios normales de uso. La métrica objetivo-académica es una disponibilidad mayor o igual al 99% en el entorno de despliegue.
- Escalabilidad gradual: sostener el crecimiento de usuarios y operaciones sin rediseño completo. La métrica objetivo es poder aumentar la capacidad del sistema mediante ajuste de configuración y evolución incremental de componentes.

4.3 Restricciones de la Arquitectura

4.3.1 Restricciones Tecnológicas

El proyecto está condicionado por las siguientes decisiones tecnológicas ya implementadas en el repositorio:

- Frontend SPA desarrollado con React y Vite en TypeScript. Backend con NestJS en TypeScript, organizado en módulos por dominio funcional. Autenticación y autorización mediante JWT local con Passport JWT y control de acceso por roles. Persistencia mediante Prisma ORM con configuración actual sobre SQLite. Pipeline de integración continua configurado con GitHub Actions, que ejecuta pruebas del backend, pruebas E2E y compilación del frontend. Configuración de despliegue para Render, con un servicio web para el backend y un sitio estático para el frontend.

Estas decisiones limitan la incorporación inmediata de tecnologías no presentes en el código actual, como proveedores externos de identidad o arquitectura de microservicios distribuidos.

4.3.2 Restricciones de Tiempo y Presupuesto

El proyecto tiene alcance académico y tiempo limitado por el calendario del curso. El equipo de desarrollo es reducido, por lo que se prioriza la simplicidad y los entregables funcionales sobre la complejidad infraestructural. El presupuesto está orientado a herramientas gratuitas o de bajo costo, priorizando un despliegue práctico y demostrable sobre infraestructura empresarial compleja.

4.3.3 Restricciones de Rendimiento

El sistema debe responder de forma fluida para operaciones CRUD y consulta de agenda en condiciones normales de uso. La exportación de citas a CSV debe completarse sin bloquear la experiencia del usuario de forma prolongada, dado que la generación ocurre en memoria en el servidor backend y se entrega directamente al navegador mediante la cabecera HTTP Content-Disposition.

4.3.4 Restricciones Legales y Reglamentarias

Debe evitarse la exposición de datos sensibles de pacientes. El sistema debe implementar control de acceso por rol para proteger operaciones clínicas y administrativas. Debe existir trazabilidad mínima de acciones críticas sobre citas, implementada actualmente mediante el modelo HistorialCita que registra reagendamientos. Para un entorno de producción real se requeriría fortalecimiento adicional de seguridad, cifrado de datos en tránsito y en reposo, y cumplimiento de normativas de protección de datos en salud.

4.3.5 Restricciones Operativas y de Entorno

El sistema debe poder ejecutarse localmente por el equipo técnico y el docente sin infraestructura compleja. Debe poder desplegarse en servicios de nube de bajo costo para demostración. La arquitectura debe permitir evolución posterior, como la migración de SQLite a PostgreSQL o la incorporación de un proveedor de identidad externo, sin reescritura total del sistema.

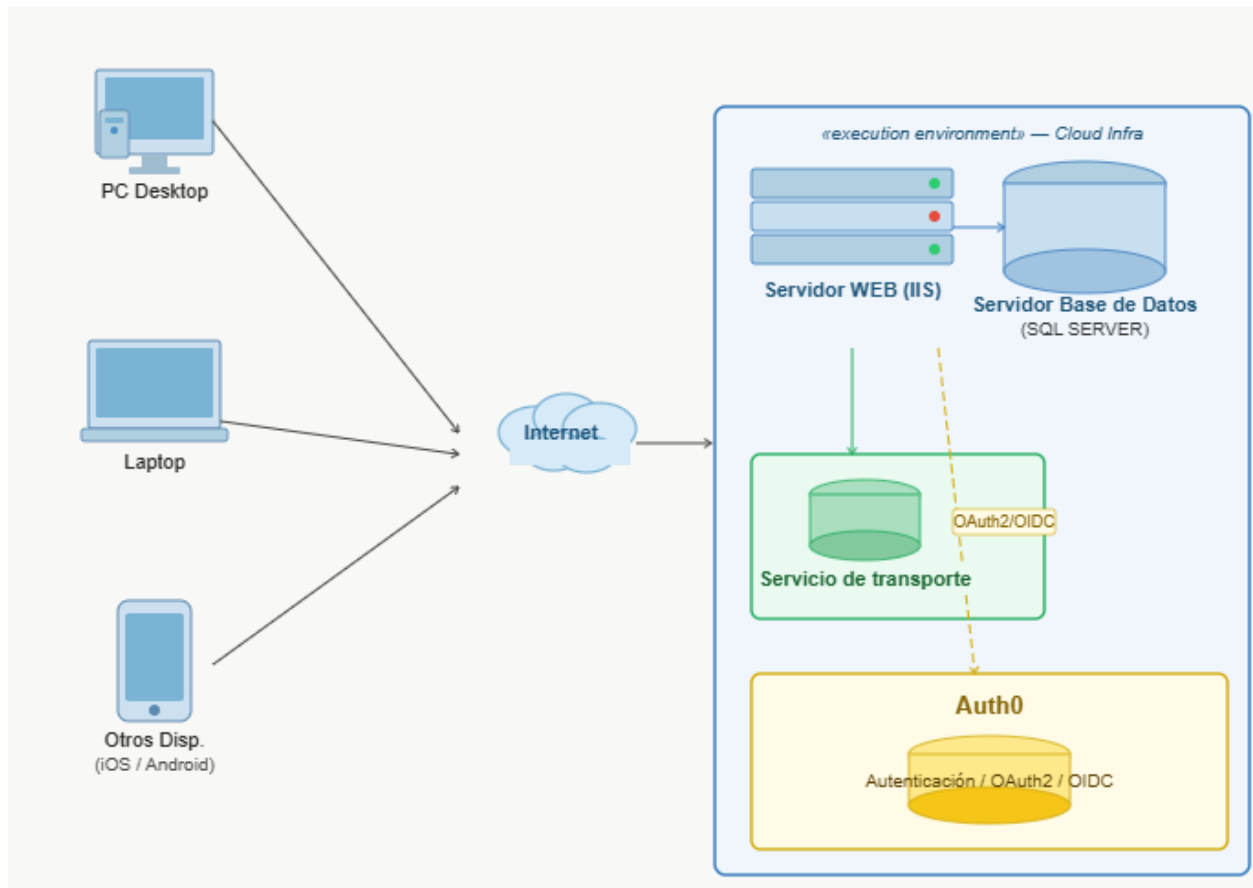
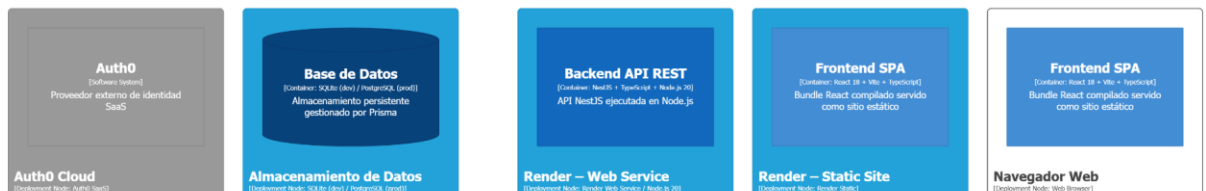


Diagrama de despliegue preliminar

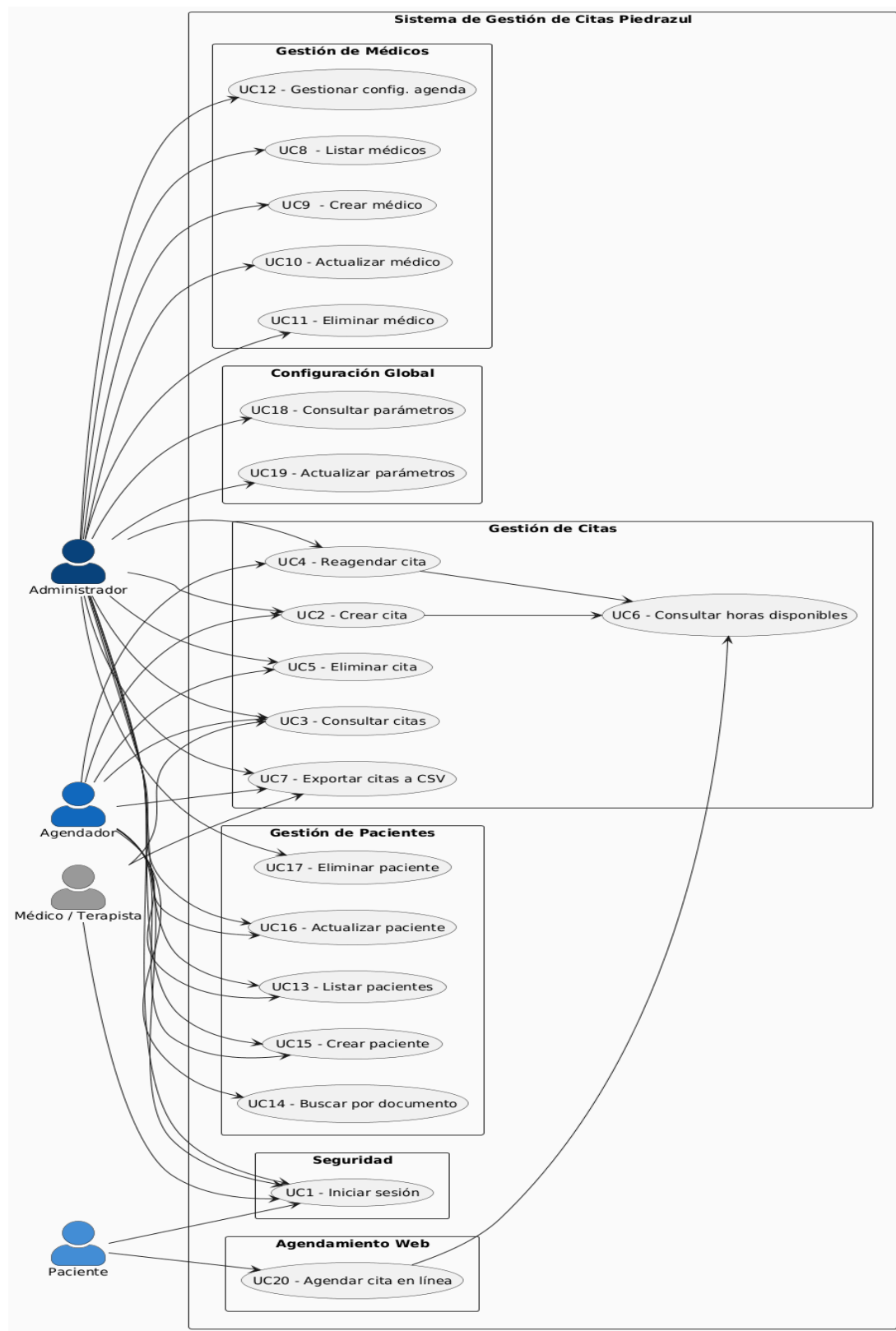


C4 – Vista de Despliegue en Render + Auth0
domingo, 3 de mayo de 2026, 11:00 a. m. hora estándar de Colombia

4.4 Vista de Casos de Uso y Escenarios de Calidad

Esta sección describe los casos de uso prioritarios y los escenarios de calidad que influyen directamente en las decisiones arquitectónicas del sistema.

4.4.1 Modelo de Casos de Uso



El modelo de casos de uso del sistema de gestión de citas de Piedrazul identifica tres actores principales y los subsistemas funcionales con los que interactúan.

Los actores del sistema son los siguientes:

- Administrador: usuario con privilegios de configuración que gestiona médicos, configuración global del sistema y tiene acceso a todas las operaciones disponibles.
- Agendador: personal administrativo que crea, consulta, reagenda y exporta citas para pacientes que solicitan atención por canales externos.
- Paciente: usuario que accede al módulo de agendamiento web para solicitar citas de forma autónoma, sin necesidad de intermediarios.

Los casos de uso principales agrupados por subsistema son los siguientes. Subsistema de Seguridad: Iniciar sesión. Subsistema de Gestión de Citas: Crear cita, consultar citas, reagendar cita, eliminar cita, consultar horas disponibles, exportar citas a CSV. Subsistema de Gestión de Médicos: Listar médicos, crear médico, actualizar médico, eliminar médico, consultar y actualizar configuración de agenda del médico. Subsistema de Gestión de Pacientes: Listar pacientes, buscar paciente por documento, crear paciente, actualizar paciente, eliminar paciente. Subsistema de Configuración Global: Consultar y actualizar parámetros globales de agendamiento. Subsistema de Agendamiento Web: Agendar cita como paciente en línea.

La tabla siguiente presenta los casos de uso más relevantes para la arquitectura y el criterio de selección.

ID	Nombre	Criterio de Selección
CU1	Autenticación: Iniciar sesión	Alto impacto transversal: todos los demás flujos dependen de la autenticación. Involucra JWT, validación de rol y redirección por perfil.
CU2	Servicios de Citas: Registrar cita	Alta frecuencia de uso. Involucra validación de disponibilidad, reglas de horario del médico y persistencia. Flujo central del sistema.
CU3	Servicios de Citas: Reagendar cita	Flujo crítico por el registro de historial de cambios. Involucra validación de disponibilidad, trazabilidad y consistencia de datos.
CU4	Servicios de Citas::Exportar citas a CSV	Impacto en rendimiento y control de acceso por rol. Generación dinámica de archivo en memoria en el servidor.
CU5	Servicios de Administración: Gestionar médicos, pacientes y configuración	Escenarios administrativos clave. Su correcta implementación afecta la disponibilidad y coherencia de los datos operativos.

Tabla 1. Casos de uso más relevantes y criterios de selección.

4.4.2 Especificación de los Casos de Uso Relevantes

ID	CU1
Nombre	Iniciar sesión
Actores	Administrador, Agendador, Paciente
Sinopsis	El usuario ingresa sus credenciales al sistema. Si son válidas, el backend genera un token JWT que el frontend almacena localmente para autenticar peticiones posteriores. Según el rol del usuario, el sistema redirige a la vista correspondiente.
Curso Típico de Eventos	
	1. El usuario ingresa nombre de usuario y contraseña en el formulario de login. 2. El sistema envía las credenciales al endpoint /auth/login. 3. El backend valida las credenciales contra la base de datos mediante bcrypt. 4. El sistema genera un token JWT con el rol del usuario y lo devuelve al frontend. 5. El frontend almacena el token y redirige según el rol: administrador a configuración, agendador a citas, paciente a agendamiento web
Extensiones	
	1.a. Campos incompletos: El sistema valida en frontend y muestra mensaje de campos obligatorios. 2.a. Error de conexión: Falla la comunicación con /auth/login; se notifica al usuario. 3.a. Error interno:Falla en validación backend; se responde con error 500. 4.a. Error en token: No se genera o es inválido; se solicita reintentar autenticación.
Prioridad AQ	Alta

ID	CU2
Nombre	Servicios de Citas :: Registrar cita
Actores	Administrador, Agendador
Sinopsis	El usuario autorizado registra una nueva cita para un paciente con un médico específico. El sistema valida disponibilidad, restricciones de horario e intervalos antes de persistir la cita
Curso Típico de Eventos	
	1. El usuario selecciona el paciente por documento o lo registra si no existe. 2. El usuario selecciona el médico y la fecha deseada. 3. El sistema consulta las horas disponibles según la configuración del médico. 4. El usuario selecciona la franja horaria disponible e ingresa el motivo de consulta. 5. El sistema valida que la franja no esté ocupada y que cumpla con los intervalos configurados. 6. El sistema registra la cita con estado AGENDADA y confirma la operación.
Extensiones	
	3.a. No hay franjas disponibles para la fecha seleccionada: el sistema informa al usuario. 5.a. La franja fue tomada por otra solicitud concurrente: el sistema informa y recarga la disponibilidad.
Prioridad AQ	Alta

ID	CU3
Nombre	Servicios de Citas :: Reagendar cita
Actores	Agendador, Administrador
Sinopsis	El usuario modifica la fecha y hora de una cita existente. El sistema valida la nueva disponibilidad, aplica el cambio y registra automáticamente el reagendamento en el historial de la cita.
Curso Típico de Eventos	
	1. El usuario selecciona una cita existente desde el listado. 2. El usuario ingresa la nueva fecha y hora en el modal de reagendamento. 3. El sistema consulta las horas disponibles para el médico en la nueva fecha. 4. El sistema valida que la nueva franja no esté ocupada y que sea posterior a la fecha actual. 5. El sistema actualiza la cita y registra el cambio en el historial con el usuario que lo realizó, el campo modificado y los valores anterior y nuevo.
Extensiones	
	3.a. La franja seleccionada no está disponible: el sistema informa y presenta las opciones disponibles. 4.a. La cita no existe: el sistema responde con error 404.
Prioridad AQ	Alta

ID	CU4
Nombre	Servicios de Citas :: Exportar citas a CSV
Actores	Agendador, Administrador
Sinopsis	El usuario descarga un reporte de citas filtrado por médico y fecha en formato CSV para organizar la jornada de atención.
Curso Típico de Eventos	
	1. El usuario selecciona el médico y la fecha como filtros. 2. El sistema consulta las citas correspondientes en la base de datos. 3. El sistema genera el archivo CSV en memoria con columnas: ID Cita, Paciente, Documento, Celular, Médico, Especialidad, Fecha, Hora, Estado, Motivo. 4. El sistema entrega el archivo al navegador mediante la cabecera Content-Disposition..
Extensiones	
	2.a. No hay citas para el filtro seleccionado: el sistema informa que no hay registros y no genera archivo. 3.a. Error al generar el archivo: el sistema responde con código de error e informa al usuario.
Prioridad AQ	Alta

ID	CU5
Nombre	Servicios de Administración :: Gestionar médicos, pacientes
Actores	Agendador (según módulo), Administrador
Sinopsis	El usuario autorizado ejecuta operaciones de consulta, creación, actualización y eliminación sobre médicos, pacientes y parámetros de configuración global del sistema.
Curso Típico de Eventos	
	1. El usuario accede al módulo correspondiente según su rol. 2. El usuario consulta el listado de registros existentes. 3. El usuario crea, actualiza o elimina un registro. 4. El sistema valida los datos ingresados. 5. El sistema persiste los cambios y confirma la operación.
Extensiones	
	3.a. Los datos ingresados no son válidos: el sistema muestra mensajes de error descriptivos. 3.b. El usuario no tiene el rol requerido para la operación: el sistema responde con error 403.
Prioridad AQ	Alta

4.4.3 Escenarios de Calidad Relevantes

A continuación, se describen los escenarios de calidad principales derivados del análisis de requisitos y las necesidades de los stakeholders. Los escenarios seleccionados están asociados a los atributos de Seguridad, Usabilidad y Escalabilidad, que son los de mayor impacto arquitectónico para el sistema.

- **A nivel de Seguridad:** el escenario seleccionado es relevante porque el sistema maneja información clínica y operativa sensible. Garantizar que ningún endpoint crítico sea accesible sin autenticación y autorización válida es la base de confianza del sistema tanto para el administrador como para el agendador.
- **A nivel de Usabilidad:** el flujo de agendamiento debe ser completado de forma fluida por usuarios con distintos niveles de experiencia tecnológica. La eficiencia del flujo principal impacta directamente la adopción del sistema por parte de los usuarios operativos.
- **A nivel de Escalabilidad:** el diseño modular y el despliegue desacoplado del frontend y el backend permiten que el sistema soporte un incremento progresivo de operaciones sin requerir un rediseño completo de la arquitectura.

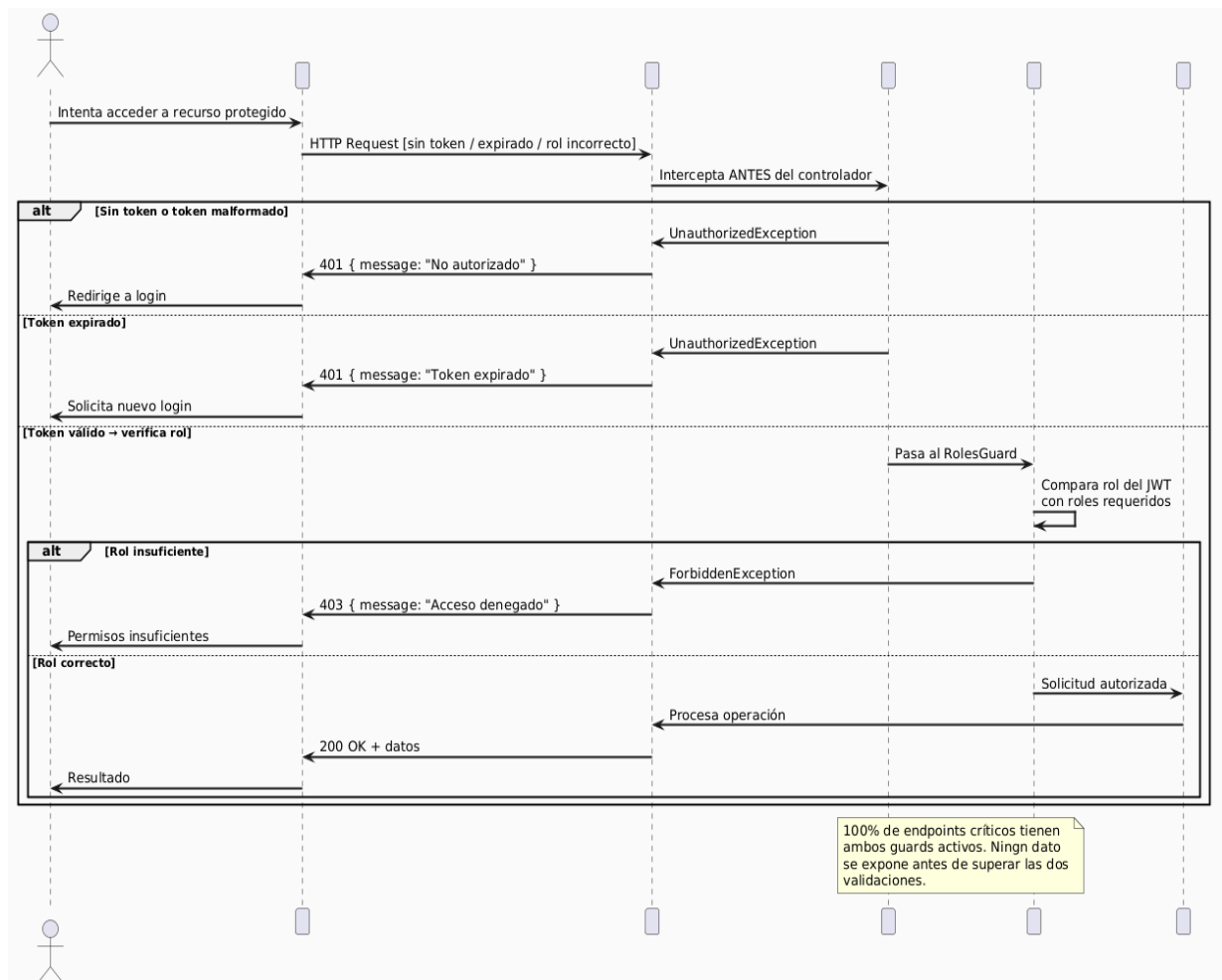


Diagrama de escenario de seguridad

ID: QS1

Nombre: Seguridad: Acceso no autorizado a operación protegida

Sinopsis: Un usuario intenta ejecutar una operación sobre citas, médicos o configuración sin token JWT válido o sin el rol requerido por la operación.

Entorno: El sistema se encuentra operando normalmente. El backend NestJS tiene activos los guarda de autenticación JWT y control de roles en todos los endpoints protegidos.

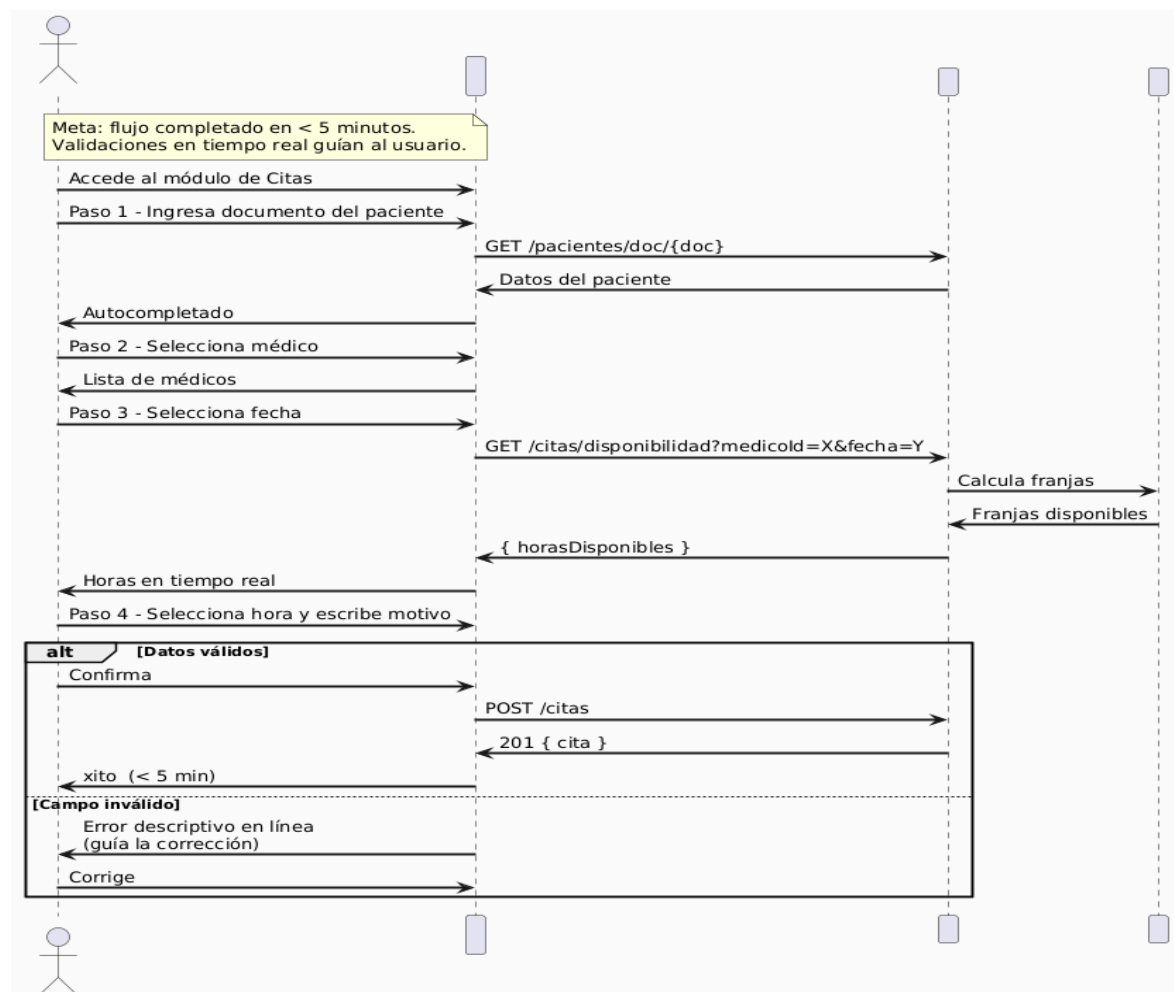
Cambio en el entorno: Se realiza una solicitud HTTP a un endpoint protegido sin cabecera de autorización, con token expirado o con un rol que no tiene permisos para la operación.

Comportamiento esperado: El guard de autenticación o el guard de roles intercepta la solicitud antes de llegar al controlador y responde con error HTTP 401 o 403 según el caso. Ningún dato del sistema es expuesto.

Medida: El 100% de las operaciones protegidas deben rechazar accesos no autorizados. La tasa de falsos positivos (accesos legítimos rechazados) no debe superar 1 en 100.000 peticiones.

Prioridad Arquitectónica: Alta

Aplicación: Local y despliegue en nube



ID: QS2

Nombre: Usabilidad: Tiempo de ejecución del flujo de agendamiento

Sinopsis: Un usuario operativo debe completar el flujo de creación o reagendamiento de una cita sin dificultades, guiado por validaciones claras y mensajes comprensibles.

Entorno: El sistema se encuentra en funcionamiento normal. El usuario tiene acceso autenticado y el módulo de citas está disponible.

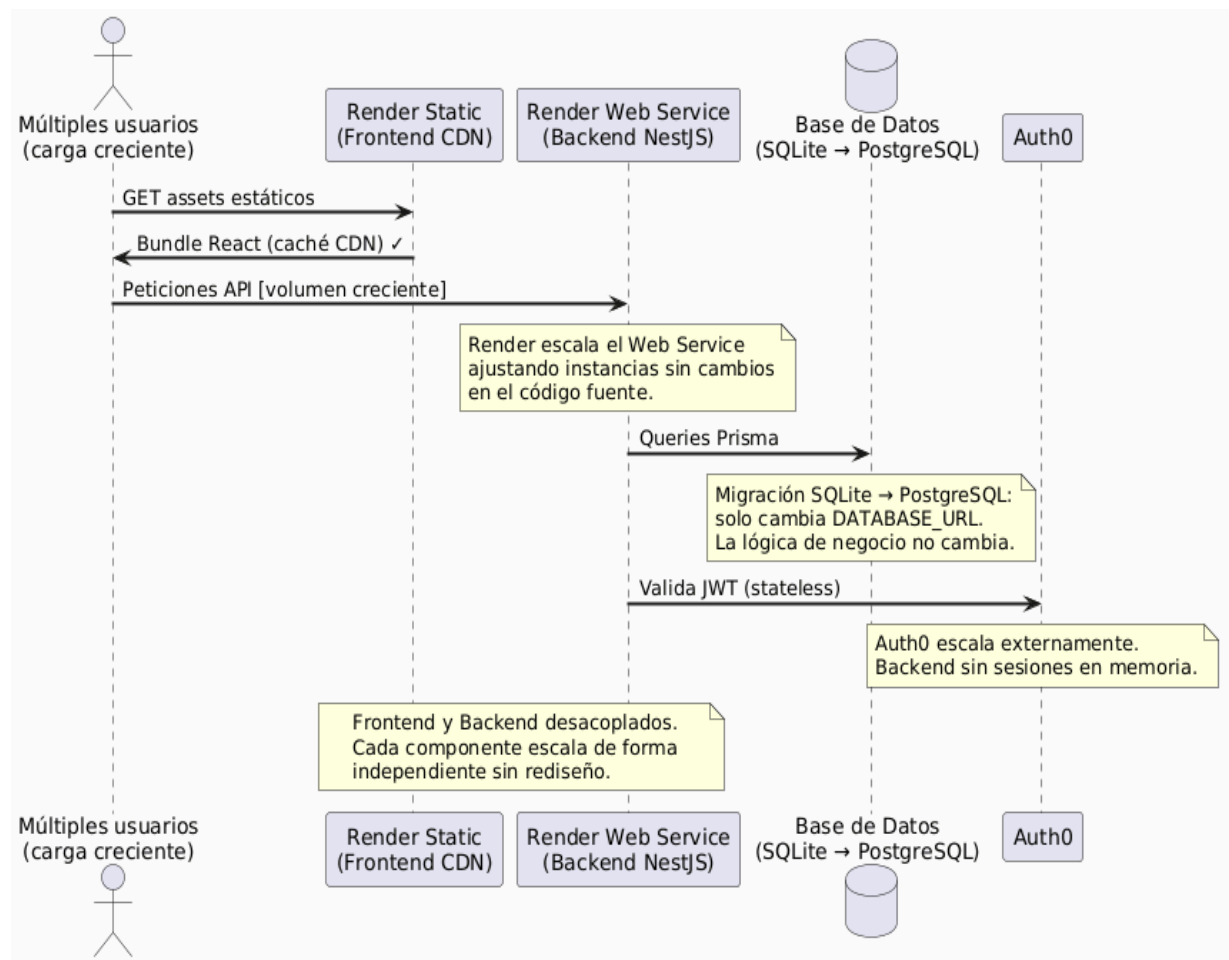
Cambio en el entorno: Un usuario nuevo o con poca experiencia intenta completar el flujo de registro o reagendamiento de una cita.

Comportamiento esperado: El sistema guía el flujo con validaciones en tiempo real, mensajes de error descriptivos y presentación clara de las horas disponibles. El usuario puede completar la operación sin asistencia técnica.

Medida: Completar el flujo principal de agendamiento en menos de cinco minutos. La tasa de abandono del flujo por errores de validación debe ser inferior al 10%.

Prioridad Arquitectónica: Alta

Aplicación: Local y despliegue en nube



ID: QS3

Nombre: Escalabilidad : Incremento de carga transaccional

Sinopsis: Aumenta el número de usuarios activos y el volumen de operaciones sobre citas, médicos y pacientes a medida que la clínica crece.

Entorno: El sistema se encuentra estable en operación normal, desplegado en Render con el frontend y el backend separados.

Cambio en el entorno: Se produce un incremento sostenido de consultas y escrituras sobre el sistema.

Comportamiento esperado: El sistema mantiene estabilidad y tiempos de respuesta aceptables mediante ajuste incremental de la configuración de despliegue. La separación entre frontend estático y backend permite escalar cada componente de forma independiente.

Medida: Degradación controlada y continuidad del servicio sin rediseño total inmediato. La migración de SQLite a PostgreSQL debe poder realizarse sin modificar la lógica de negocio, aprovechando el desacoplamiento provisto por Prisma.

Prioridad Arquitectónica: Alta

Aplicación: Despliegue en nube

4.4.4 Métricas y Prioridades de Calidad

Las métricas de seguimiento propuestas para evaluar el cumplimiento de los atributos de calidad son las siguientes. Para Seguridad: porcentaje de endpoints críticos con protección activa y tasa de intentos no autorizados correctamente bloqueados. Para Usabilidad: tiempo promedio de completar el flujo de agenda y tasa de errores por validación de formulario. Para Rendimiento: tiempo de respuesta promedio en operaciones de consulta y escritura, y tiempo de generación de exportaciones CSV. Disponibilidad: porcentaje de tiempo de servicio disponible en el entorno desplegado. La prioridad relativa de los atributos de calidad es la siguiente: primero Seguridad, segundo Usabilidad, tercero Disponibilidad, cuarto Modificabilidad y quinto Escalabilidad.

5 Vista Lógica

La vista lógica describe la organización interna del sistema desde una perspectiva funcional y conceptual. En el sistema de gestión de citas de Piedrazul, esta vista muestra cómo se estructuran el frontend, el backend, la autenticación, la persistencia y los módulos de negocio, así como la forma en que estos elementos colaboran para cumplir los requisitos funcionales.

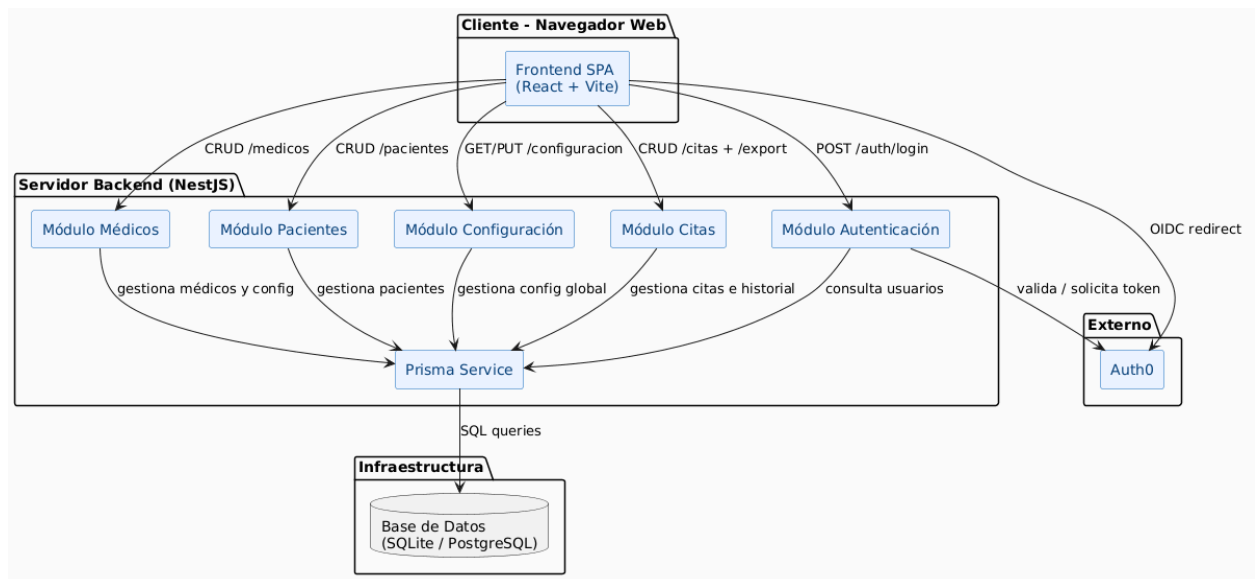
5.1 Parte Estructural: descomposición modular

El sistema se organiza en módulos coherentes y de alta cohesión. La estructura lógica principal distingue claramente entre la interfaz de usuario, la lógica de aplicación, la seguridad y el acceso a datos. Los módulos principales identificados en el repositorio son los siguientes:

- Frontend SPA en React + Vite.
- Módulo de autenticación.
- Módulo de citas.

- Módulo de médicos.
- Módulo de pacientes.
- Módulo de configuración.
- Capa de persistencia con Prisma.

Esta separación permite que cada módulo tenga una responsabilidad específica y que el sistema evolucione sin afectar innecesariamente a los demás.

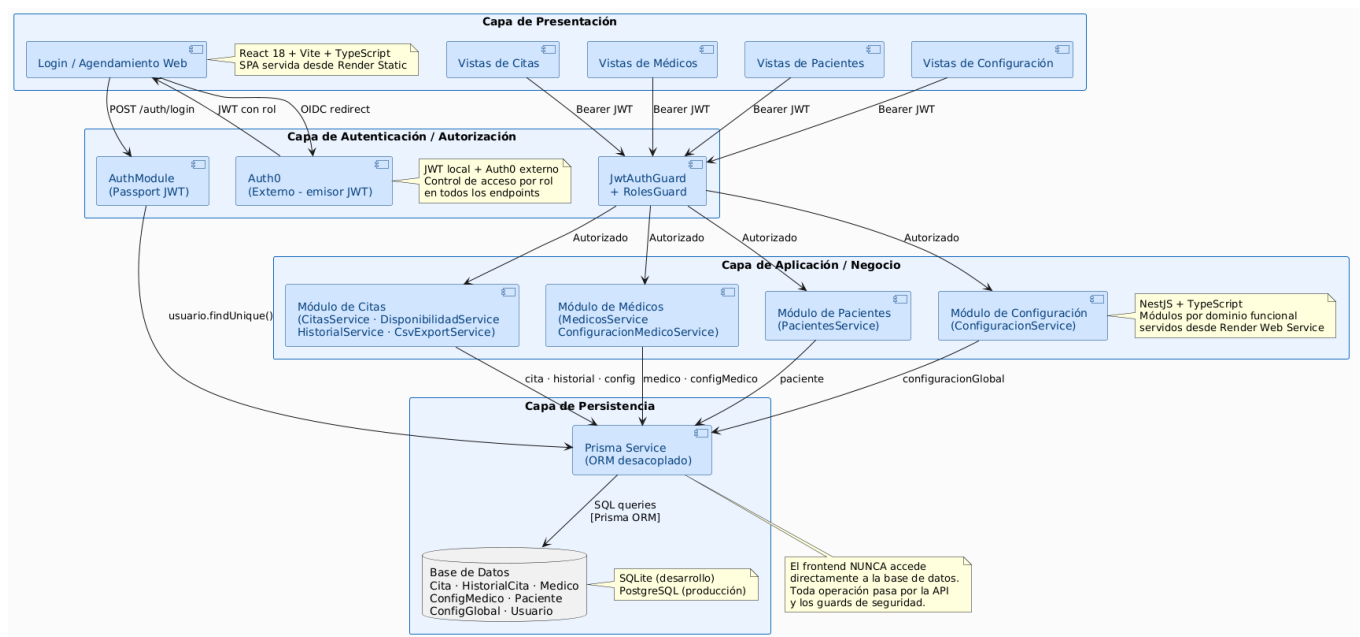


5.1.1 Primer Nivel de Refinamiento

En un primer nivel de refinamiento, el sistema puede entenderse como la composición de cuatro subsistemas lógicos principales:

Subsistema	Descripción	Responsabilidad principal
Presentación	Interfaz web del usuario	Mostrar las vistas de citas, médicos, pacientes, configuración y login
Aplicación / Negocio	API NestJS	Recibir solicitudes, aplicar reglas de negocio y coordinar operaciones
Autenticación y Autorización	JWT y roles	Validar credenciales, emitir tokens y restringir acceso por roles
Persistencia	Prisma + base de datos	Guardar y consultar usuarios, médicos, pacientes, citas y configuración

Este primer nivel responde al estilo cliente-servidor, donde el frontend consume la API REST del backend y no accede directamente a la base de datos.



Primer nivel de refinamiento

5.1.2 Segundo Nivel de Refinamiento

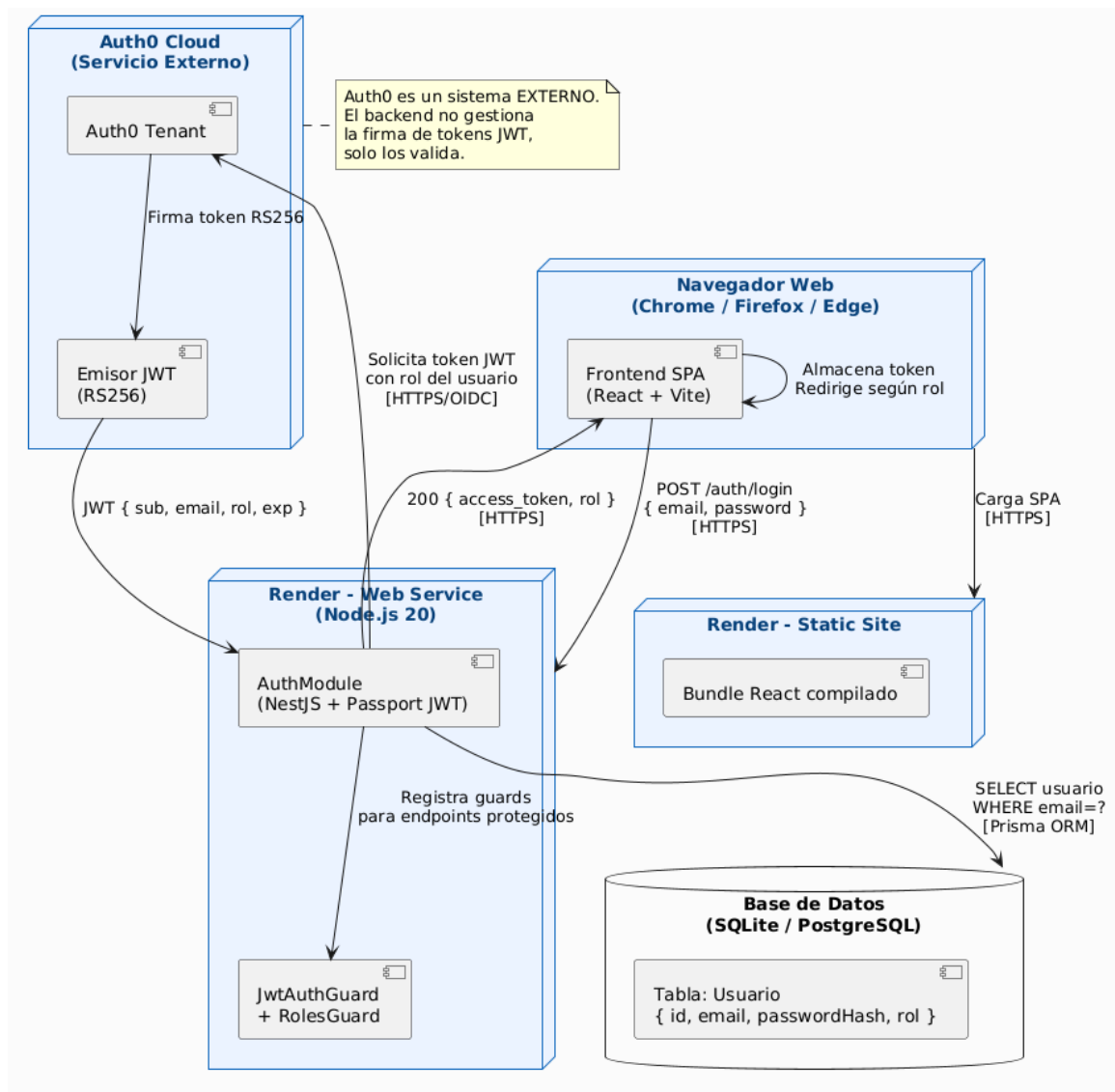
El segundo nivel descompone el backend en módulos más específicos. En el proyecto actual, los módulos del backend están alineados con los casos de uso principales del sistema.

5.1.2.1 Módulo de Autenticación

Este módulo se encarga de validar usuarios, generar JWT y proteger el acceso a los recursos. Usa Passport JWT para extraer el token desde la cabecera Authorization y validar las peticiones entrantes.

Responsabilidades:

- Validar credenciales.
- Emitir token JWT.
- Controlar el acceso a rutas protegidas.
- Asignar identidad y rol al usuario autenticado.



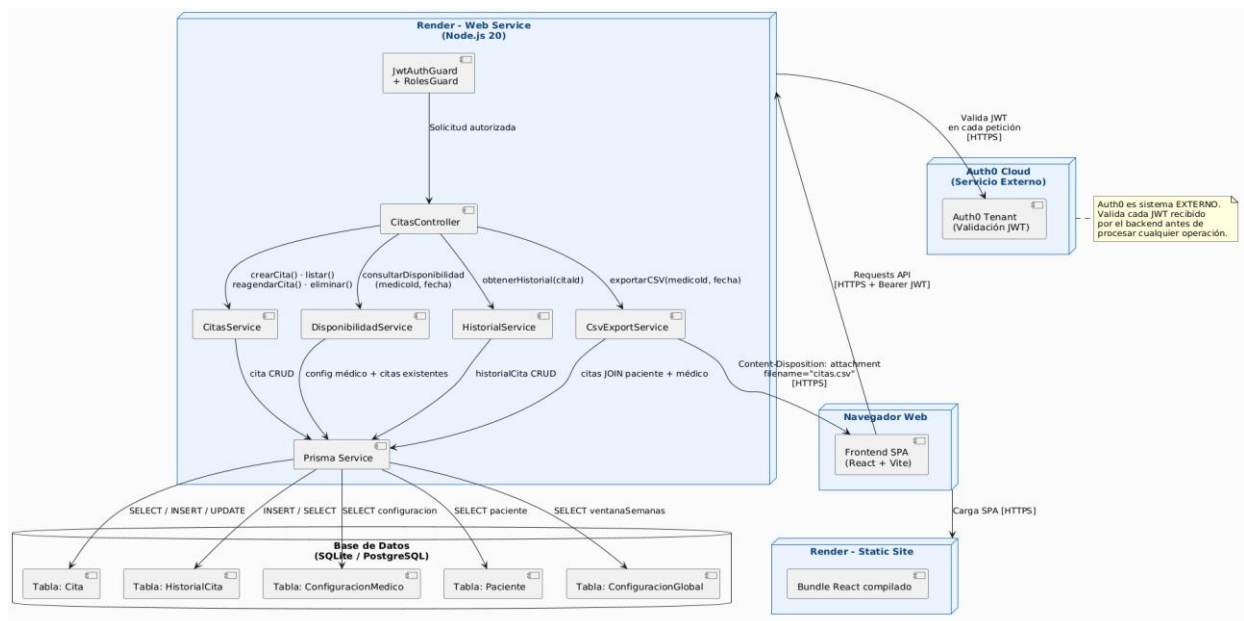
Módulo de autenticación

5.1.2.2 Módulo de Citas

Este módulo concentra la lógica relacionada con la gestión de citas. Desde aquí se administran las operaciones de consulta, creación, actualización, reagendamiento, eliminación y exportación de citas.

Responsabilidades:

- Listar citas con filtros y orden.
- Consultar disponibilidad.
- Registrar y modificar citas.
- Reagendar citas.
- Generar exportación CSV.
- Restringir operaciones según rol.
- Reagendar citas validando disponibilidad y registrando el cambio en el historial.
- Consultar el historial de modificaciones de una cita específica.



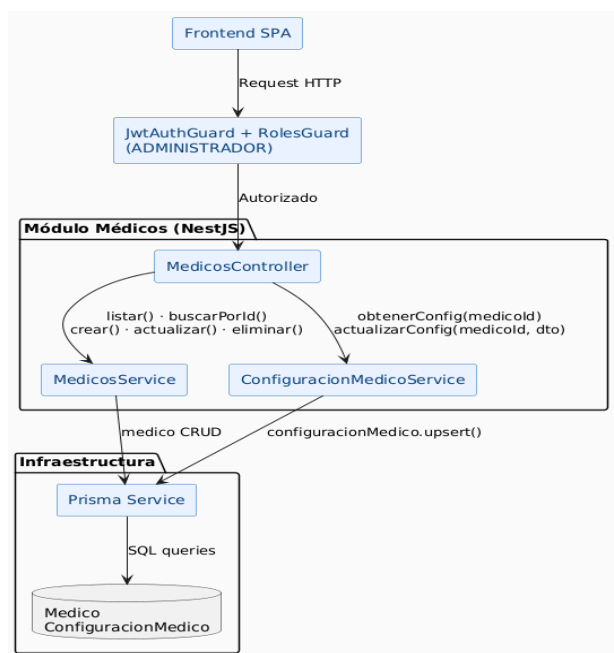
Descomposición del subsistema citas

5.1.2.3 Módulo de Médicos

Este módulo gestiona la información de los médicos y su configuración asociada.

Responsabilidades:

- Listar médicos.
- Consultar un médico por identificador.
- Crear, actualizar y eliminar médicos.
- Consultar y actualizar la configuración de agenda de cada médico (días de atención, horario de inicio y fin, intervalo entre citas), gestionada desde los endpoints `/médicos/:id/configuración`.

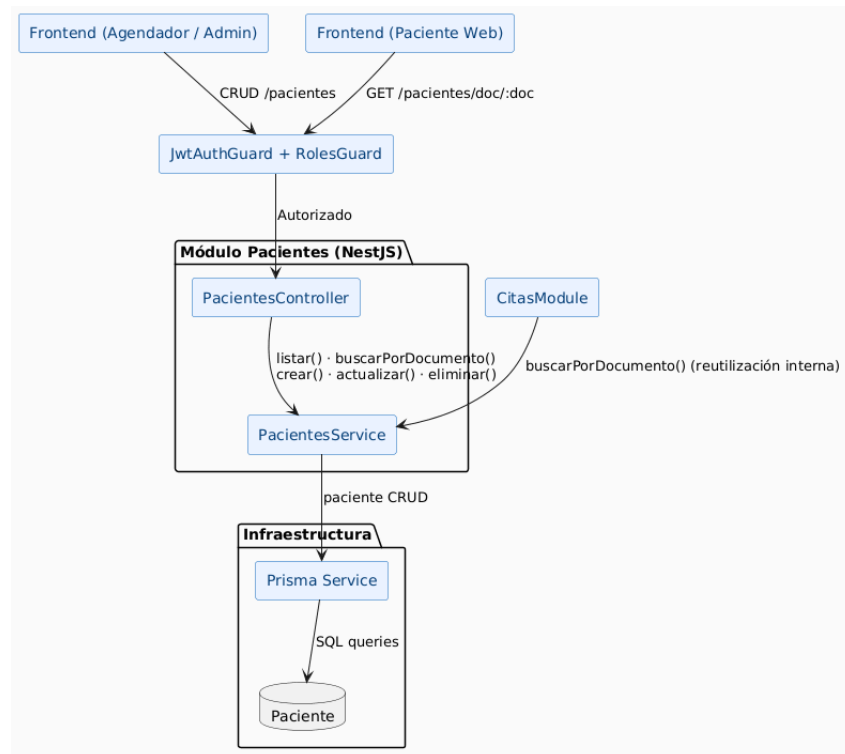


Descomposición del subsistema Médicos

5.1.2.4 Módulo de Pacientes

Este módulo administra el registro y mantenimiento de pacientes. Responsabilidades:

- Listar pacientes.
- Buscar pacientes por documento.
- Crear, actualizar y eliminar pacientes.



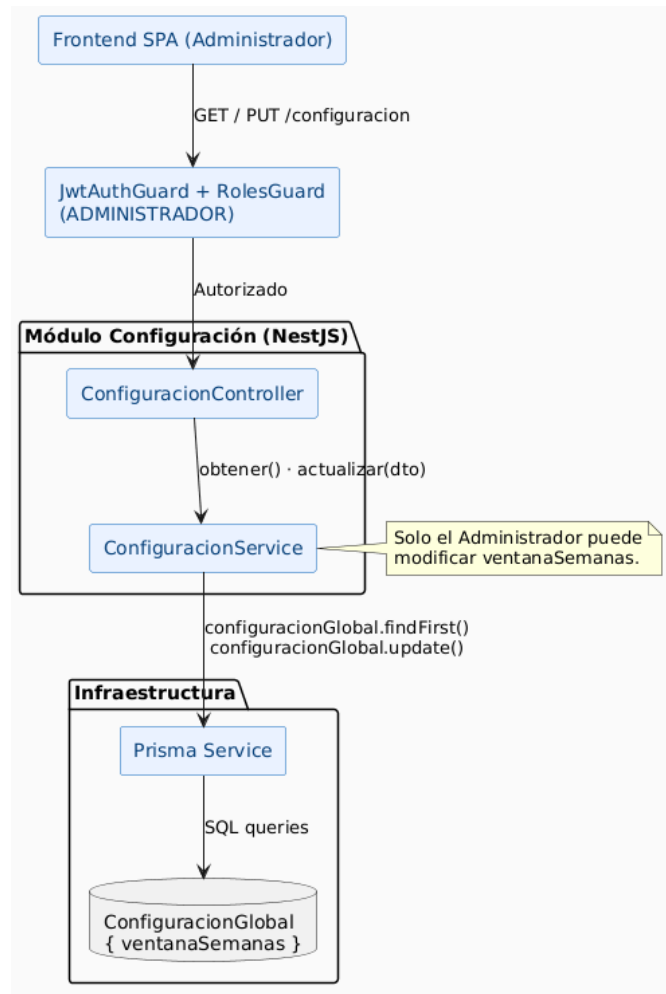
Módulo de pacientes

5.1.2.5 Módulo de Configuración

Este módulo administra únicamente la configuración global del sistema, representada por el modelo Configuración Global. El único parámetro actual es ventana Semanas, que define cuántas semanas hacia adelante se habilita el agendamiento. La configuración individual de cada médico (días, horarios, intervalos) es responsabilidad del módulo de médicos.

Responsabilidades:

- Consultar configuración global.
- Actualizar configuración global.



Descomposición del subsistema Configuración

5.1.2.6 Capa de Persistencia

La persistencia está desacoplada de la lógica de negocio mediante Prisma. El backend accede a la base de datos a través de un servicio centralizado de Prisma, evitando que los controladores o servicios conozcan detalles de implementación del motor de almacenamiento.

Responsabilidades:

- Ejecutar consultas de lectura y escritura.
- Administrar usuarios, médicos, pacientes, citas y configuración.
- Mantener separada la lógica de acceso a datos de la lógica funcional.

5.2 Parte Dinámica (Comportamiento)

La vista dinámica describe cómo interactúan los componentes durante la ejecución del sistema. En el proyecto actual, los escenarios más representativos son el inicio de sesión, la gestión de citas, la consulta de disponibilidad, la administración de pacientes y médicos, y la exportación de información en CSV.

5.2.1 Escenario de Autenticación

El usuario ingresa sus credenciales desde el frontend. La aplicación envía la solicitud al backend, donde se validan los datos contra la base persistida por Prisma. Si las credenciales son correctas, el backend genera un token JWT que el frontend almacena localmente para invocar rutas protegidas. Una vez recibido el token, el frontend extrae el rol del usuario y redirige automáticamente a la vista correspondiente: el administrador accede a la configuración, el agendador a la gestión de citas, y el paciente al módulo de agendamiento web en línea.

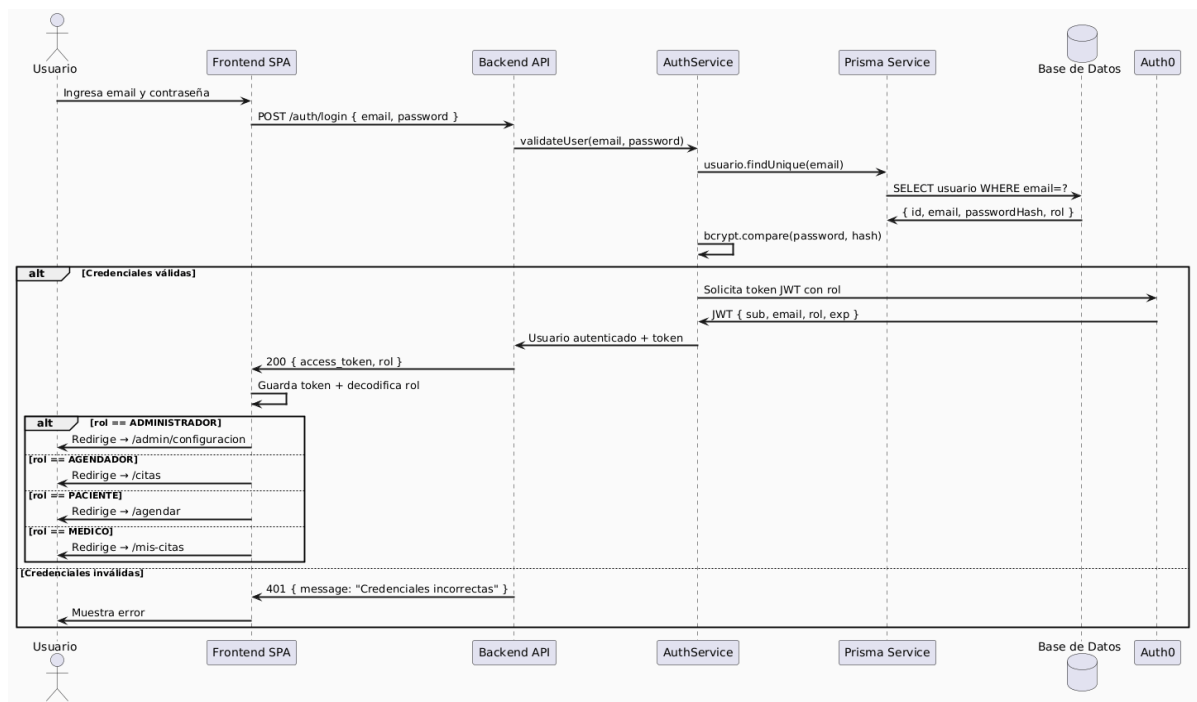


Diagrama de secuencia inicio de sesión

5.2.2 Escenario de Gestión de Citas

Una vez autenticado, el usuario autorizado consulta las citas, revisa disponibilidad, crea nuevas citas o modifica citas existentes. El backend aplica control de acceso por roles y valida cada operación antes de persistir el cambio. Adicionalmente, el usuario puede reagendar una cita existente seleccionando desde el listado. El sistema presenta un modal con los campos de nueva fecha y hora, consulta las horas disponibles en tiempo real y valida las restricciones de horario antes de confirmar el cambio. Cada reagendamiento queda registrado automáticamente en el historial de la cita, permitiendo consultar quién realizó el cambio, qué campo se modificó y cuáles fueron los valores anterior y nuevo.

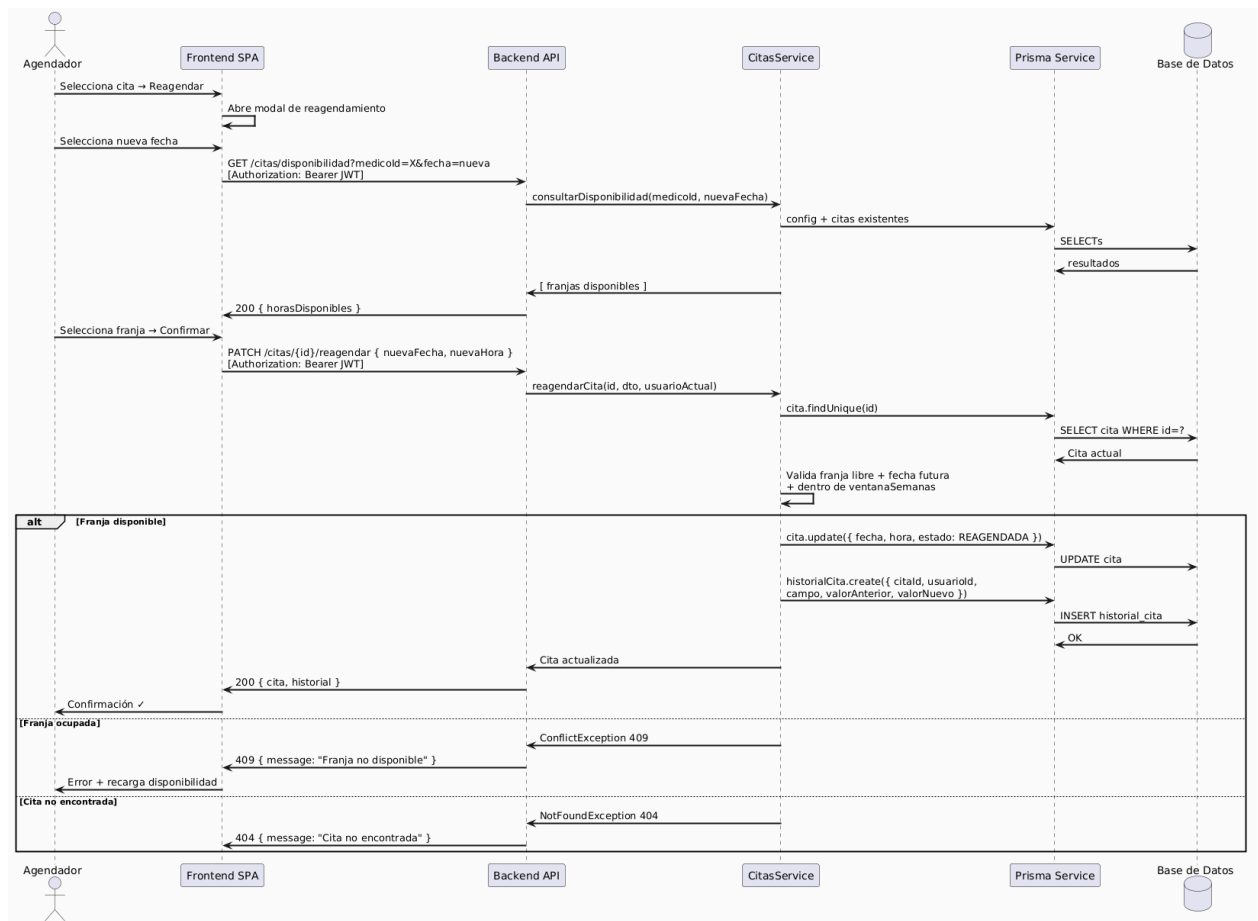


Diagrama de secuencia gestión de cita

5.2.3 Escenario de Exportación CSV

El usuario autorizado solicita la exportación de citas. El backend genera el archivo CSV en memoria y responde con la cabecera Content-Disposition para forzar la descarga en el navegador.

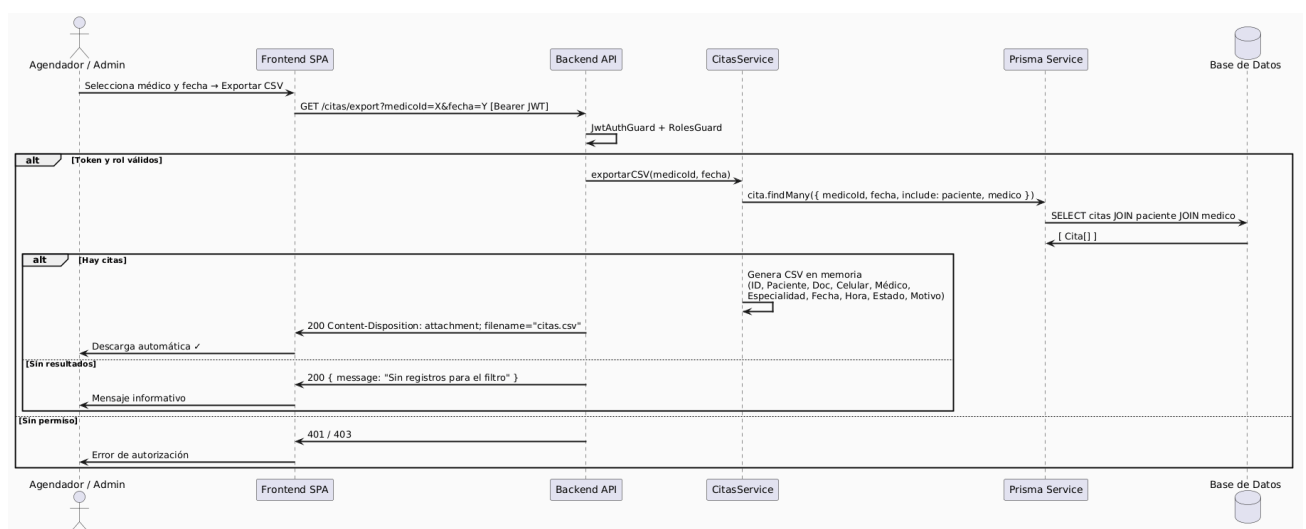
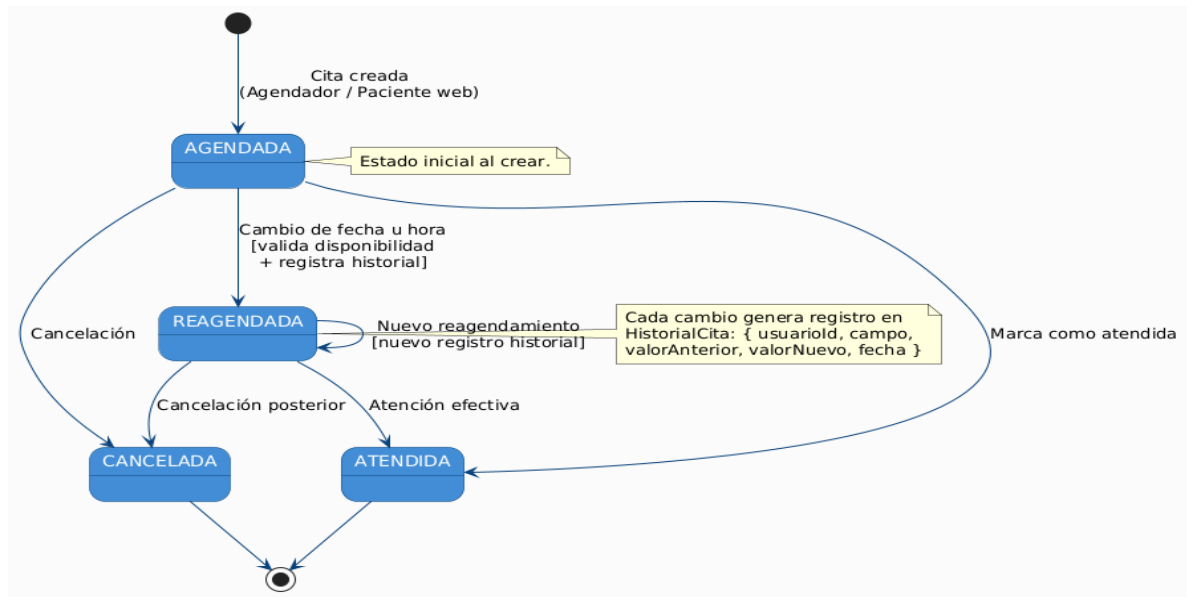


Diagrama de secuencia gestión de cita

5.2.4 Escenario de Administración de Médicos, Pacientes y Configuración

Los módulos administrativos permiten consultar, registrar, editar y eliminar información según el rol del usuario. La lógica de negocio y las validaciones se concentran en el backend, mientras que el frontend solo presenta la interfaz y consume la API.



Escenario de administración

5.3 Organización de los Componentes

La organización lógica del sistema sigue una separación clara entre presentación, negocio, autenticación y persistencia:

- El frontend React muestra las vistas y captura las acciones del usuario.
- Los servicios del frontend encapsulan las llamadas HTTP al backend.
- El backend NestJS expone controladores REST para cada dominio funcional.
- Los servicios del backend implementan reglas de negocio.
- Prisma centraliza la persistencia.
- JWT y los guardas controlan el acceso a los recursos protegidos.

Esta organización favorece la mantenibilidad, la testabilidad y la extensión futura. Las reglas más importantes observadas en el código actual son las siguientes:

- Solo los usuarios autenticados pueden acceder a la mayor parte de los recursos.
- Algunas operaciones sobre citas están restringidas a roles específicos.
- La exportación de citas solo se habilita para usuarios autorizados.
- La configuración global controla parámetros que afectan la agenda.
- La gestión de pacientes y médicos está desacoplada, pero ambos módulos alimentan el flujo de agendamiento.

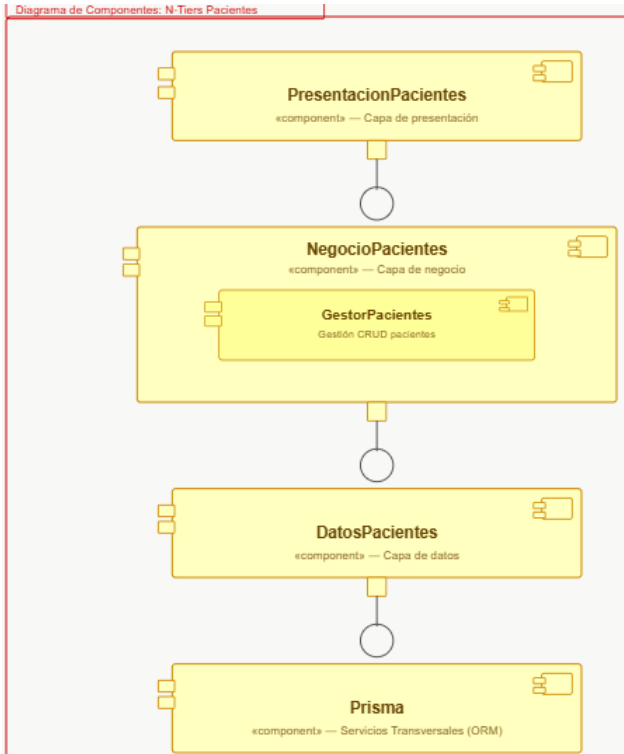
5.4 View Packets (Catálogo de Componentes)

A continuación, los view packets para **Médicos** y para **Pacientes**, que son módulos con lógica propia:

- View Packet Negocio Médicos:

Nombre del View Packet	Negocio Médicos
Rationale	El patrón Cliente-Servidor permite centralizar la lógica de gestión de médicos y su configuración de agenda en el servidor. Al concentrar en un único módulo tanto la información del médico como su configuración de disponibilidad, se evita la dispersión de lógica y se garantiza que cualquier cambio en los parámetros de agenda sea consistente con el resto del sistema.
Componente	Gestor de médicos
Descripción:	Componente encargado de las operaciones CRUD sobre médicos y de la gestión de su configuración de agenda individual (días de atención, franja horaria, intervalo entre citas).
Composición Interna	Diagrama de Componentes: N-Tiers Medicos El diagrama muestra la estructura de componentes en capas: PresentacionMedicos («componente» — Capa de presentación) se conecta a NegocioMedicos. NegocioMedicos («componente» — Capa de negocio) contiene dos sub-componentes: GestorMedicos (Gestión CRUD médicos) y GestorConfiguracion Medico (Config. horarios / especialidades). NegocioMedicos se conecta a DatosMedicos. DatosMedicos («componente» — Capa de datos) se conecta a Prisma. Prisma («componente» — Servicios Transversales (ORM)) es el componente base de la capa de datos.

- *View Packet Negocio Pacientes:*

Nombre del View Packet	Negocio Pacientes
Rationale	<i>La separación del módulo de pacientes respecto al de citas permite que la búsqueda por documento y el registro de nuevos pacientes sean operaciones independientes y reutilizables desde cualquier flujo de agendamiento, ya sea interno (agendador) o externo (paciente web). Esto reduce el acoplamiento entre módulos.</i>
Componente	Gestor de pacientes
Descripción:	<i>Componente encargado del registro, búsqueda por número de documento, actualización y eliminación de pacientes. Provee autocompletado de datos tanto al módulo de citas del agendador como al módulo de agendamiento web del paciente.</i>
Composición Interna	 Diagrama de Componentes: N-Tiers Pacientes <pre> graph TD subgraph NP [NegocioPacientes «component» — Capa de negocio] GP[GestorPacientes Gestión CRUD pacientes] end PP[PresentacionPacientes «component» — Capa de presentación] DP[DatosPacientes «component» — Capa de datos] P[Prisma «component» — Servicios Transversales (ORM)] PP --> NP NP --> DP DP --> P </pre>

6 Vista de Despliegue

La vista de despliegue describe cómo los componentes del sistema se distribuyen y ejecutan en la infraestructura física o virtualizada. En el caso del sistema de gestión de citas de Piedrazul, la implementación actual corresponde a una arquitectura web cliente-servidor, donde el frontend se ejecuta como una SPA y consume una API REST desarrollada en el backend.

6.1 Nodos y Componentes Desplegados

El sistema de Piedrazul se compone de los siguientes nodos de despliegue:

Nodo	Componentes Desplegados	Protocolo de Comunicación
ClienteWeb (Navegador)	SPA Frontend en React + Vite: vistas de citas, médicos, pacientes, configuración y agendamiento web	HTTP/HTTPS hacia el backend
ServidorBackend (Aplicación)	API REST desarrollada con NestJS: módulos de autenticación, citas, médicos, pacientes y configuración	HTTP/REST desde el frontend
ServidorBaseDatos (Local o Entorno de Desarrollo)	Base de datos gestionada por Prisma; en el estado actual del repositorio se utiliza SQLite	Acceso local mediante Prisma
Plataforma de Despliegue	Servicio de hosting para backend y frontend estático; el repositorio incluye configuración para Render	HTTP/HTTPS

Nodos de despliegue y componentes del sistema Piedrazul.

6.2 Descripción del Flujo de Despliegue

El flujo de comunicación entre nodos opera de la siguiente manera: el usuario accede al sistema a través de un navegador web que carga la SPA del frontend. Desde la interfaz, el usuario inicia sesión mediante un formulario que envía credenciales al backend NestJS en la ruta `/auth/login`. El backend valida al usuario con JWT propio, usando Passport JWT, y devuelve un token de acceso que el frontend almacena localmente para autenticar peticiones posteriores.

Una vez autenticado, el frontend consume los servicios REST expuestos por el backend para gestionar citas, médicos, pacientes y configuración. El backend protege sus endpoints con guards basados en JWT y roles, por lo que las operaciones sensibles requieren autenticación previa. La exportación de citas a CSV se realiza desde el servidor backend y se descarga al navegador mediante la cabecera `HTTP Content-Disposition`.

En el repositorio también existe una configuración de despliegue para Render, donde se define un servicio web para el backend y un sitio estático para el frontend. Además, se incluye

un pipeline de integración continua que ejecuta pruebas del backend, pruebas E2E y compilación del frontend.

6.3 Tecnología Requerida

La siguiente tabla presenta el stack tecnológico seleccionado para el sistema:

Categoría	Tecnología / Herramienta
Sistema Operativo (Servidor)	Linux en entorno de despliegue
Lenguaje Backend	TypeScript
Framework Backend	NestJS
Lenguaje Frontend	TypeScript / JavaScript
Framework Frontend	React 18 con Vite
Base de Datos	SQLite en el estado actual del repositorio
Autenticación y Autorización	JWT local con Passport JWT y control por roles
Pruebas Unitarias	Jest
Contenerización	No hay contenedores definidos en el repositorio actual
CI/CD	GitHub Actions
Despliegue en Nube	Render
Control de Versiones	Git / GitHub
Protocolo de Comunicación	HTTP/REST/JSON
Herramientas de Desarrollo	VS Code, npm

7 Rationale

El diseño arquitectónico del sistema de gestión de citas de Piedrazul se definió a partir de un enfoque incremental y guiado por los atributos de calidad que el sistema debía satisfacer. Dado que el proyecto actual está implementado como una aplicación web con frontend en React + Vite, backend en NestJS, autenticación local con JWT y persistencia con Prisma, las decisiones arquitectónicas se tomaron buscando un equilibrio entre simplicidad, modularidad, seguridad y facilidad de evolución.

7.1 Objetivos Arquitectónicos

Los objetivos principales de la arquitectura son los siguientes:

- Seguridad: proteger el acceso a los recursos del sistema mediante autenticación con JWT y autorización basada en roles.
- Modularidad: separar claramente la interfaz, la lógica de negocio, la autenticación y la persistencia.
- Mantenibilidad: permitir que cada módulo evolucione sin afectar al resto del sistema.
- Usabilidad: ofrecer una SPA simple, directa y consistente para los usuarios finales.
- Portabilidad: facilitar el despliegue independiente del frontend y del backend.
- Evolución futura: dejar abierta la posibilidad de incorporar mejoras tecnológicas sin reescribir toda la solución.

Estos objetivos son importantes porque el sistema maneja información clínica y operativa que requiere control de acceso, bajo acoplamiento y facilidad de cambio.

7.2 Decisiones Arquitectónicas Principales

7.2.1 Elección del estilo Cliente-Servidor

Se eligió el patrón Cliente-Servidor porque permite separar el consumo de la interfaz de usuario de la lógica central del sistema. El frontend actúa como cliente y el backend como servidor de servicios REST.

Esta decisión se justifica porque:

- Facilita agregar nuevos clientes en el futuro.
- Permite centralizar las reglas de negocio.
- Mejora el control de seguridad.
- Reduce la duplicación de lógica entre pantallas.

7.2.2 Elección de una SPA en React

Se adoptó una SPA en React con Vite porque el sistema necesita una interfaz rápida, modular y fácil de mantener. La aplicación actual navega entre vistas internas sin depender de una arquitectura de renderizado del lado del servidor.

Ventajas de esta elección:

- Arranque rápido durante el desarrollo.
- Composición clara por componentes.

- Facilidad para crear pantallas reutilizables.
- Menor complejidad operativa frente a una solución SSR.

Alternativas consideradas: Next.js, Angular y una solución con renderizado del lado del servidor. Se descartaron porque el repositorio actual ya está construido sobre React + Vite y no requiere, para esta versión, las ventajas adicionales de SSR o un framework más pesado.

7.2.3 Elección de NestS para el backend

Se eligió NestJS porque ofrece una estructura muy adecuada para sistemas modulares y escalables en el lado servidor. Su organización por controladores, servicios, módulos y guards se adapta bien a un sistema de citas con dominios funcionales bien delimitados.

Ventajas de esta decisión:

- Separación clara entre capas.
- Soporte natural para validación y guards.
- Integración sencilla con JWT.
- Buena mantenibilidad del código.

7.2.4 Autenticación con JWT local

En esta versión, la autenticación no se delega a un proveedor externo. El backend genera y valida sus propios tokens JWT mediante Passport JWT.

La razón principal es la simplicidad del alcance actual:

- No se requiere una federación de identidades.
- El sistema puede operar de forma autónoma.
- Se reduce la complejidad de integración.
- El control de acceso queda totalmente visible dentro del proyecto.

Alternativa considerada: Keycloak u otro proveedor externo de identidad. Se descartó en esta versión porque aumentaría el costo de configuración y no es necesario para el alcance actual del prototipo.

7.2.5 Prisma como capa de persistencia

Se eligió Prisma para aislar el acceso a datos y simplificar la interacción con la base de datos. La lógica de negocio no depende de SQL directo, sino de un servicio de persistencia controlado por el ORM.

Ventajas:

- Reduce el acoplamiento con la base de datos.
- Mejora la legibilidad del acceso a datos.
- Facilita cambios futuros del modelo.
- Favorece pruebas y mantenimiento.

7.2.6 Base de datos SQLite en el estado actual

El repositorio actual usa SQLite en el schema de Prisma. Esta decisión es coherente con un prototipo o desarrollo académico porque:

- Simplifica la instalación.
- Evita dependencias externas para correr localmente.
- Acelera la puesta en marcha.
- Facilita pruebas rápidas del sistema.

Alternativa considerada: PostgreSQL. Aunque PostgreSQL sería más adecuado para producción, en el estado actual del proyecto SQLite resulta suficiente para el alcance implementado.

7.2.7 Despliegue separado de frontend y backend

La configuración de despliegue del repositorio separa el frontend estático y el backend web. Esto simplifica la publicación de la aplicación y permite que ambos componentes evolucionen de manera independiente.

Ventajas:

- Independencia de despliegue.
- Mejor claridad operativa.
- Facilidad de escalado por componente.
- Mantenimiento más simple.

7.3 Alternativas Consideradas

Durante el diseño se evaluaron varias alternativas:

- Next.js frente a React + Vite.
- Auth0 frente a JWT local.
- PostgreSQL frente a SQLite.
- Monolito simple frente a separación modular por dominios.
- Una arquitectura con contenedores frente a una implementación más directa para despliegue académico.

La selección final prioriza la coherencia con el código ya implementado, la facilidad de mantenimiento y el menor costo de operación.

7.4 Trade-offs o Compromisos

Toda decisión arquitectónica implicó compromisos:

- Simplicidad frente a robustez empresarial: usar JWT local y SQLite simplifica el sistema, pero reduce capacidades propias de una solución productiva más grande.
- Modularidad frente a menor cantidad de código: separar por módulos mejora el mantenimiento, aunque introduce más archivos y más estructura.
- SPA frente a SSR: la SPA es más simple de operar en este contexto, pero no ofrece las ventajas completas de renderizado del lado del servidor.
- Despliegue directo frente a infraestructura más compleja: la solución actual es más fácil de mantener, pero no es la más sofisticada para alta disponibilidad.

7.5 Consideraciones del Entorno

El sistema se diseña para un entorno académico y de despliegue liviano, por lo que se tuvieron en cuenta estas restricciones:

- Facilidad de ejecución local.
- Bajo costo de infraestructura.
- Despliegue simple en servicios como Render.
- Necesidad de un backend accesible desde el navegador.
- Protección de rutas sensibles mediante autenticación.

Estas condiciones justifican el uso de una pila tecnológica ligera y un diseño modular, pero no excesivamente complejo.

7.6 Decisiones Futuras y Evolución

La arquitectura actual deja abierta la posibilidad de crecimiento futuro. Entre las evoluciones razonables se encuentran:

- Migrar SQLite a PostgreSQL.
- Reemplazar la autenticación local por un proveedor de identidad externo si el sistema crece.
- Incorporar contenedores Docker si se requiere estandarizar despliegues.
- Separar más finamente los dominios si aumentan los módulos funcionales.
- Incorporar pruebas automáticas adicionales y observabilidad.

Estas decisiones no se implementaron ahora porque el alcance actual del proyecto no lo exige, pero la estructura modular sí permite hacerlo sin rediseñar todo el sistema.

7.7 Justificación Final

En resumen, se eligió una arquitectura cliente-servidor con separación modular porque el sistema necesita una base clara, simple y mantenible para gestionar citas, pacientes, médicos y configuración. La combinación de React + Vite en el frontend, NestJS en el backend, JWT local para seguridad y Prisma para persistencia satisface los requisitos actuales del proyecto y permite una evolución ordenada hacia una solución más robusta en el futuro.

Driver Candidato	Incumbencias	Tácticas	Patrones
QS1 Seguridad	Autenticación y autorización de usuarios	Validación de token JWT en cada petición, control de acceso por roles, guards en endpoints protegidos	Cliente-Servidor, Filtro (Guard JWT con Passport), Capas
QS2 Concurrencia (doble agendamiento)	Prevención de citas duplicadas en la misma franja	Validación atómica de disponibilidad antes de registrar la cita, consulta previa al momento de creación	Cliente-Servidor, Repository (Prisma), Transacción
QS3 Disponibilidad	Recuperación del servicio ante fallo del backend	Despliegue en Render con reinicio automático, pipeline CI/CD para detección rápida de fallos	Cliente-Servidor, Despliegue en nube (Render), GitHub Actions
Modificabilidad Mantenimiento y extensión	Separación de responsabilidades por módulo	Módulos independientes en NestJS, servicios separados de controladores, Prisma desacoplado de la lógica	Capas (N-Tiers), Repository (Prisma), Módulos NestJS
Usabilidad Agendamiento eficiente	Flujo simple para el usuario	Separación de interfaz de usuario, validación en tiempo real de disponibilidad	Capas, SPA React (componentes reutilizables)
Escalabilidad Crecimiento de usuarios	Desacoplamiento entre frontend y backend	Despliegue independiente de frontend estático y backend, stateless con JWT	Cliente-Servidor, Despliegue separado en Render